Computer Architecture: Computer 101B

Tatpong Katanyukul

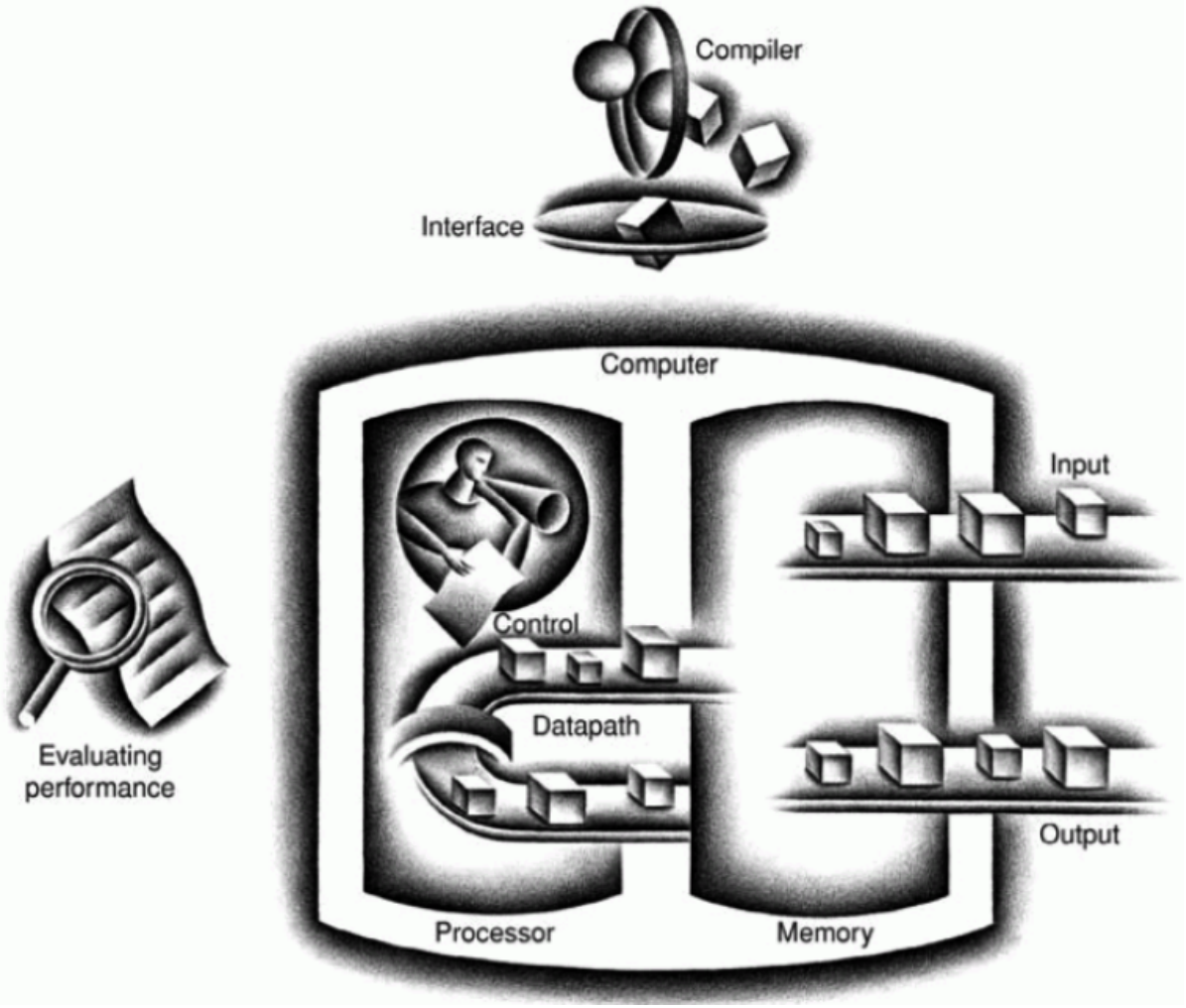
CoE, KKU

24 มกราคม พ.ศ. 2561

Previously on our discussion (1)

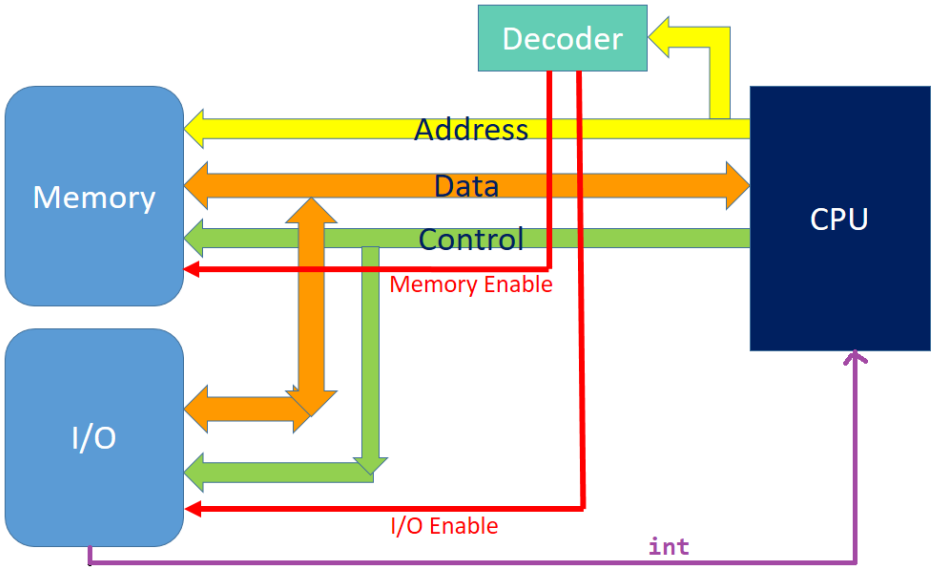
Key Components of a Computer

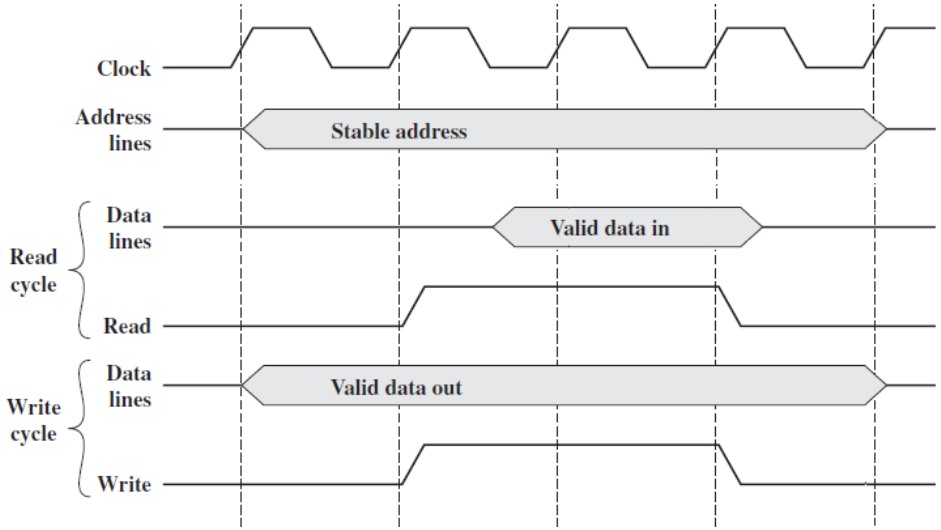
- Processor
  - Datapath
  - Control
- Memory
- I/O



Previously on our discussion (2)

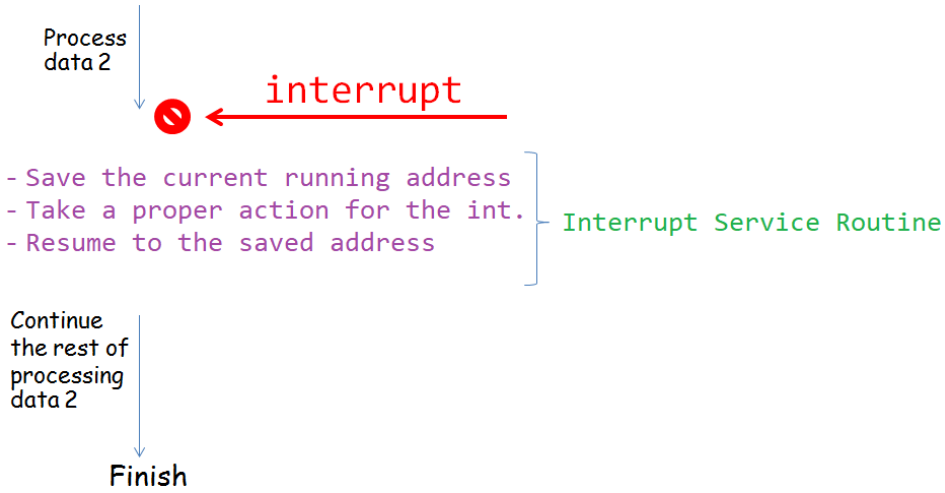
Connection and communication of CPU, memory, and I/O



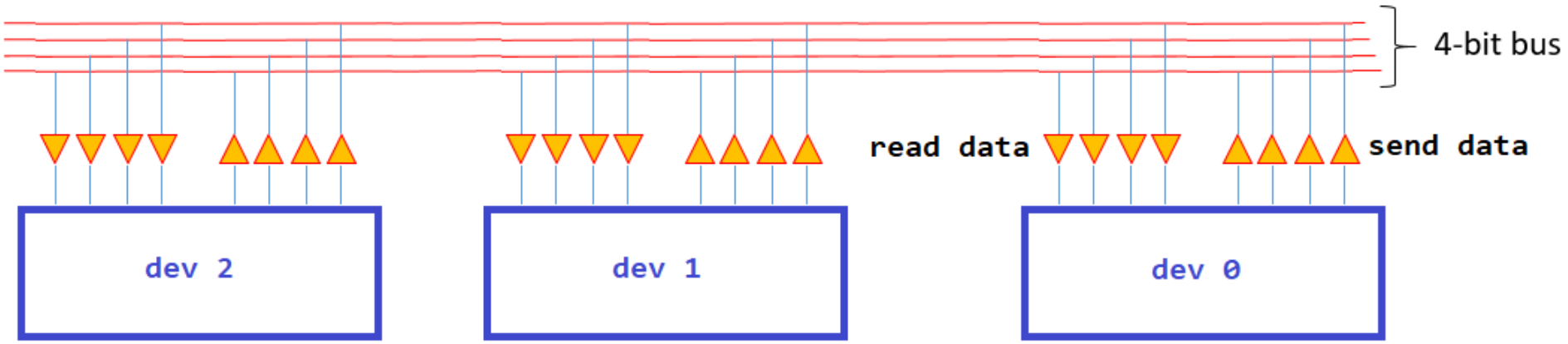


Previously on our discussion (3)

Interruption



Bus



- Bus: communication pathway connecting two or more devices
- Bus is a set of shared wires:
  data sent by one device can be read by all
- There can be only one device sending at a time

Bus: one sending at a time

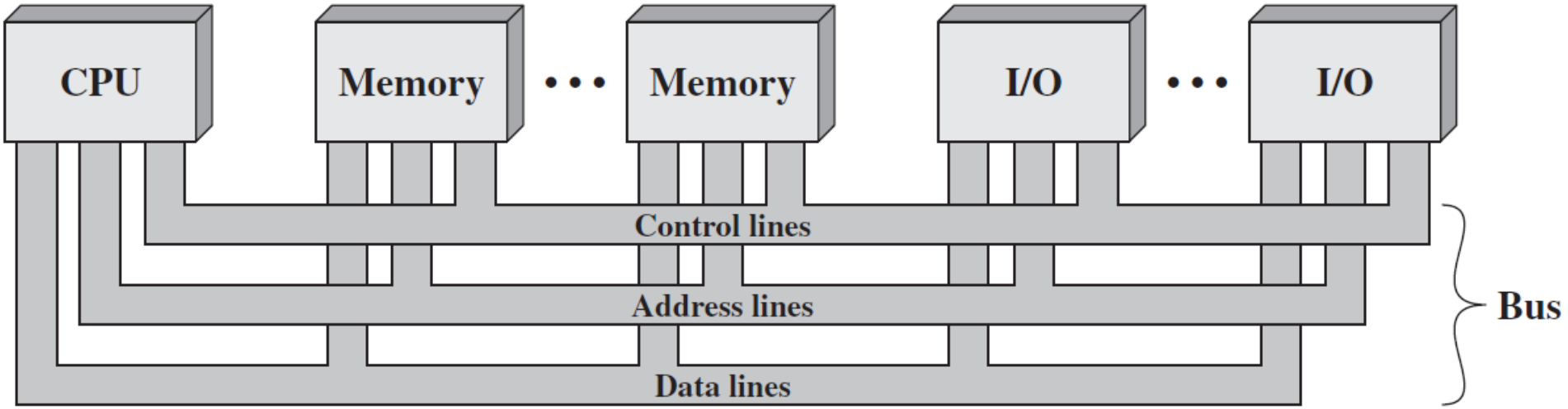


Figure 3.16 Bus Interconnection Scheme

- All sending devices are three-state logic.
- There must be a mechanism to ensure single-sender assumption. Otherwise, signals may be mixed up or, worse, it could damage the devices.

Three-state logic

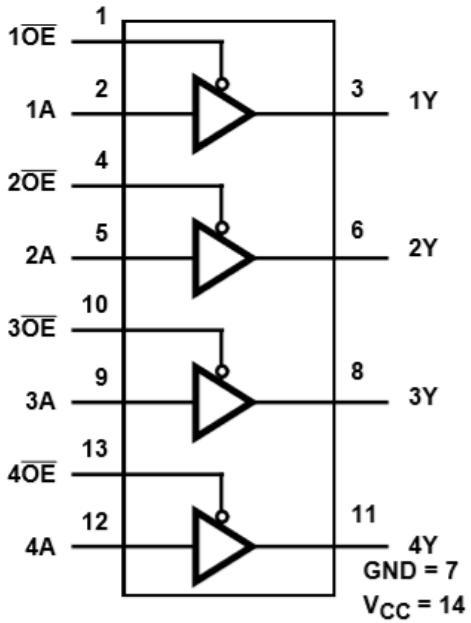
TRUTH TABLE

INPUTS		OUTPUTS
nA	nOE	nY
H	L	H
L	L	L
X	H	Z

H= High Voltage Level
L= Low Voltage Level
X= Don't Care
Z= High Impedance, OFF State

src: 74HCT125 datasheet downloaded from

http://www.ti.com/product/CD74HCT125



Bus Operations

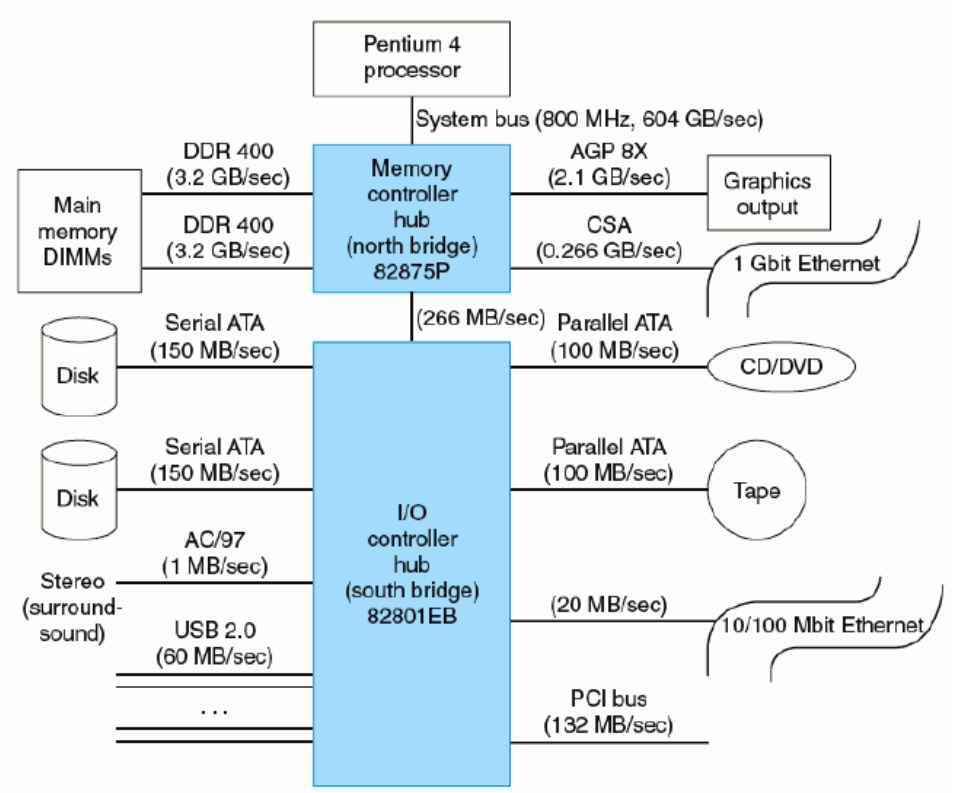
If module A wishes to send data to module B, module A must:

- (1) Obtain the grant of bus usage.
- (2) Send data on the bus.
  Address is specified to module B.

If module A wishes to request data from module B, module A must:

- (1) Obtain the grant of bus usage.
- (2) Send request on the bus.
  Address is specified to module B.
- (3) Wait for data from module B.
  Module B will signal when the data is ready.

Block Transfer



Situation: CPU works with data in a main memory. We have our data saved in our hard disk. So, we need to load our data (or program) into a main memory before we can use it.

$\Rightarrow$ Block transfer

Block Transfer

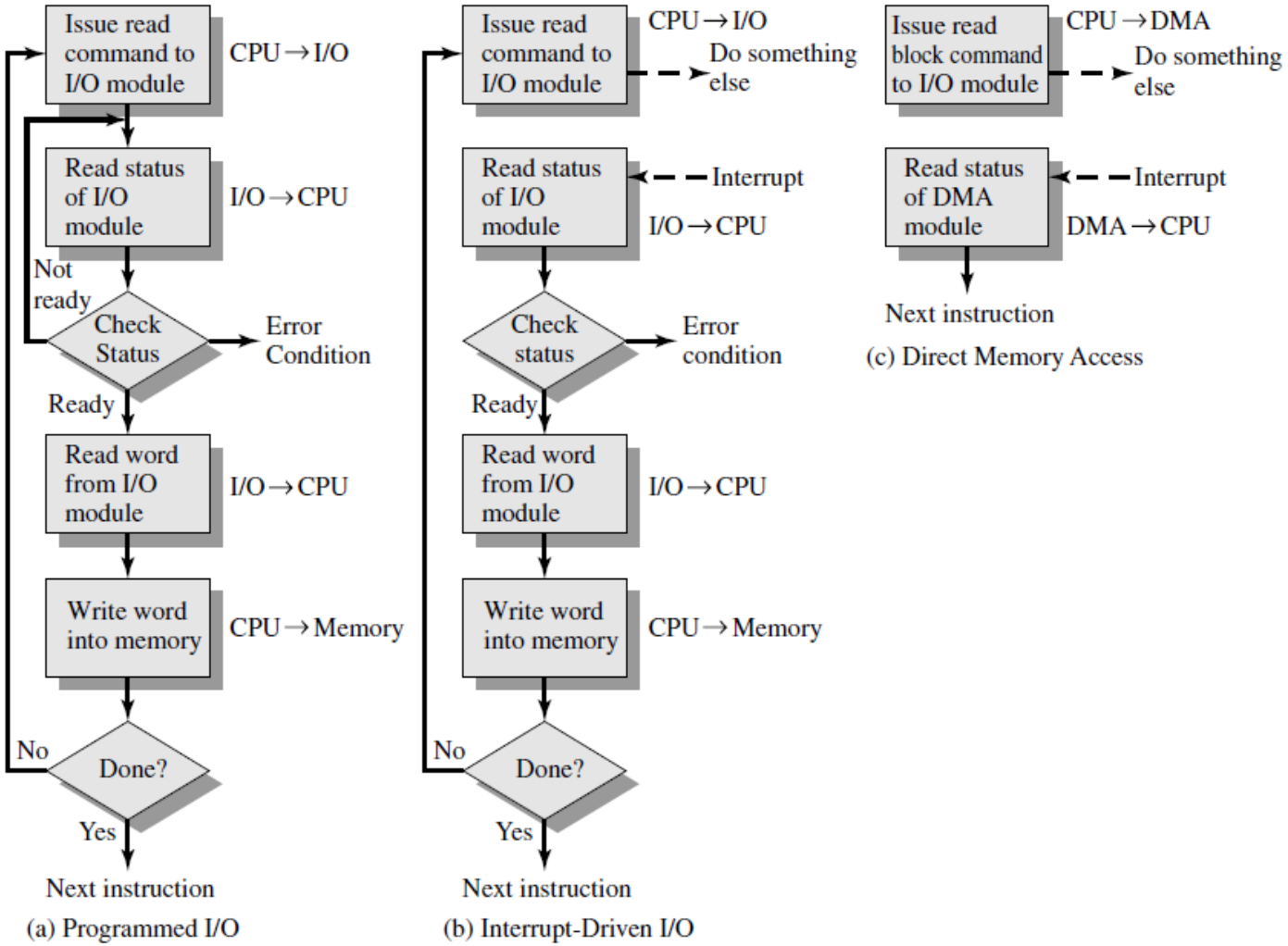
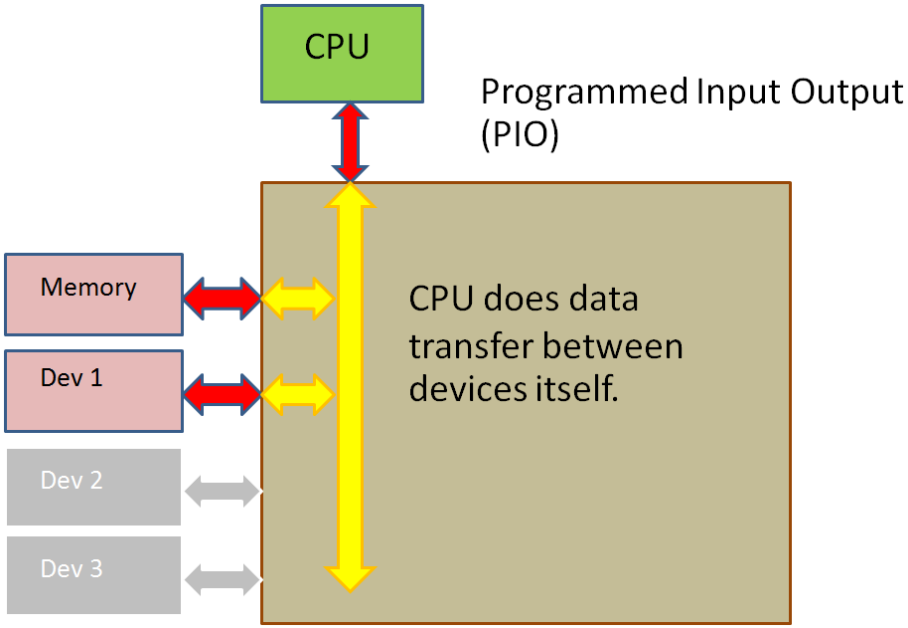


Figure 7.4 Three Techniques for Input of a Block of Data

[William Stallings, 8e]

Programmed I/O Block Transfer



- PIO transfer: CPU is tied up in managing I/O transfer.
- Interrupt-driven transfer: it frees CPU to some extent at the expense of the I/O transfer rate.

Direct Memory Access (DMA)

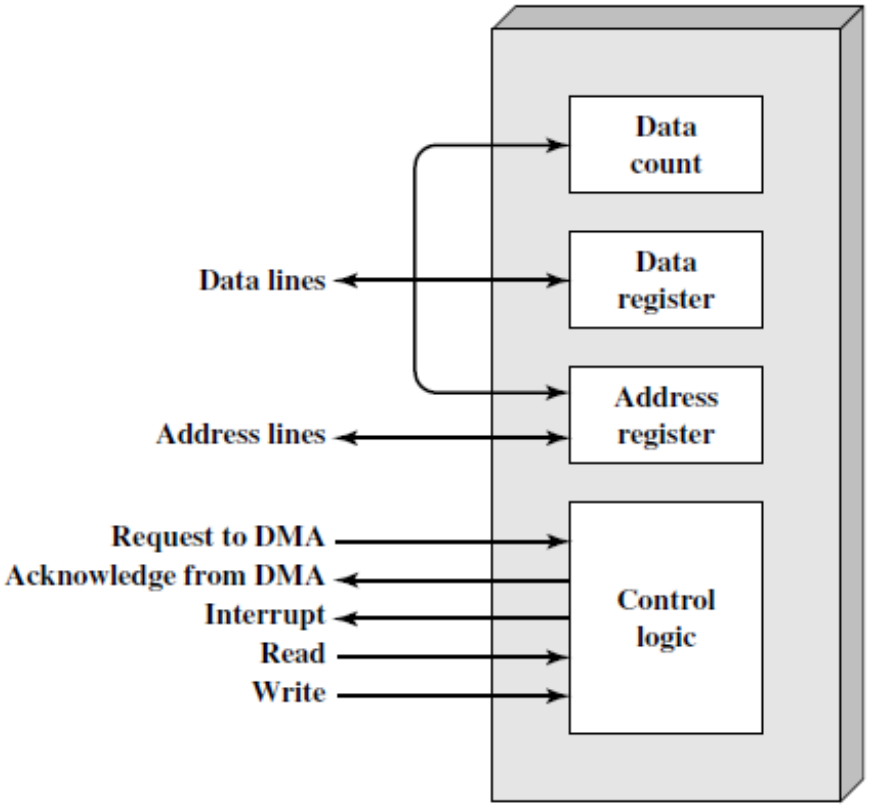
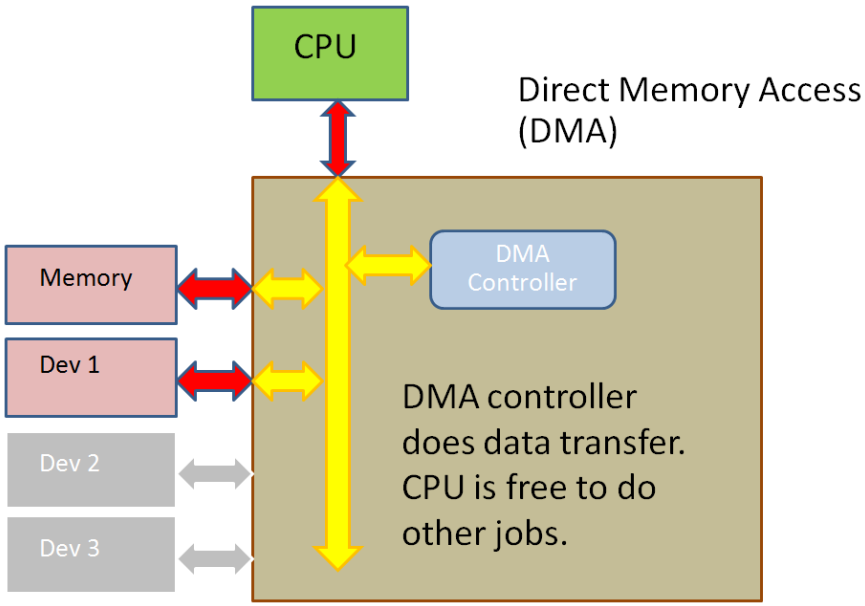


Figure 7.11 Typical DMA Block Diagram

- DMA module takes over control of the data transfer.
- DMA transfers data to and from memory over the system bus.

Hidden Cycle v.s. Cycle Stealing



DMA must be granted control of the bus.

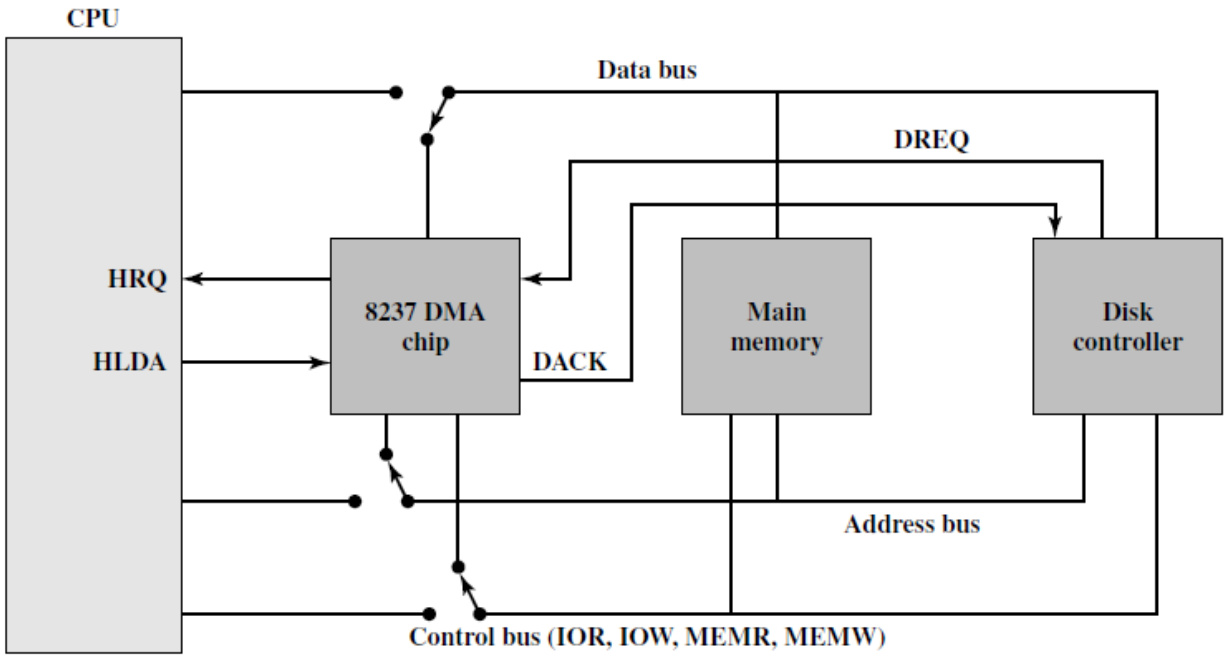
- Hidden cycle: DMA takes the bus, when CPU does not need it.
- Cycle stealing: DMA takes the bus and if CPU needs the bus, suspend CPU operation temporarily.

DMA Block Transfer

- CPU commands DMA module to do data transfer.
  $\Rightarrow$ Information for DMA
  - Direction of the transfer: read transfer (mem. to i/o) or write transfer (i/o to mem.)
  - Initial memory address
  - I/O address
  - Number of words to transfer
- CPU continues with other work.
- DMA module transfers the data.
- Once the transfer is complete, DMA module sends an interrupt signal to CPU.

DMA Controller

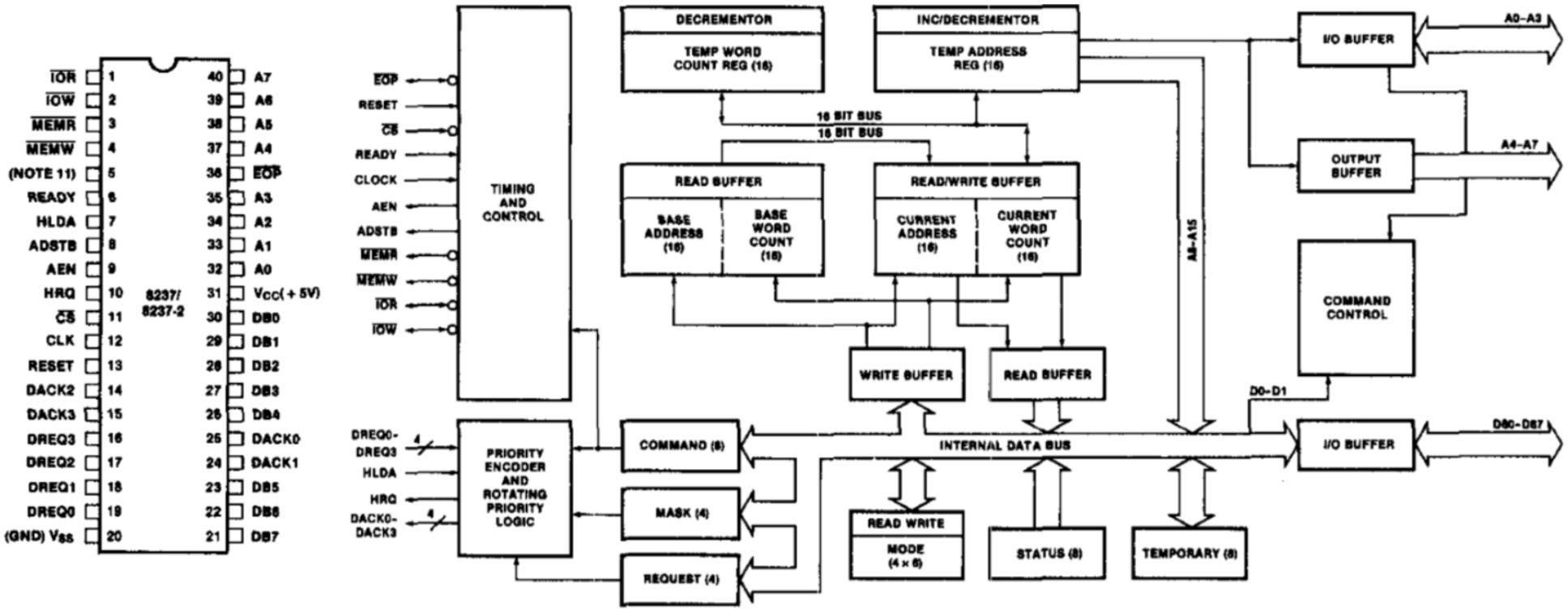
E.g., Intel 8237 DMA controller interfaces to 80x86.



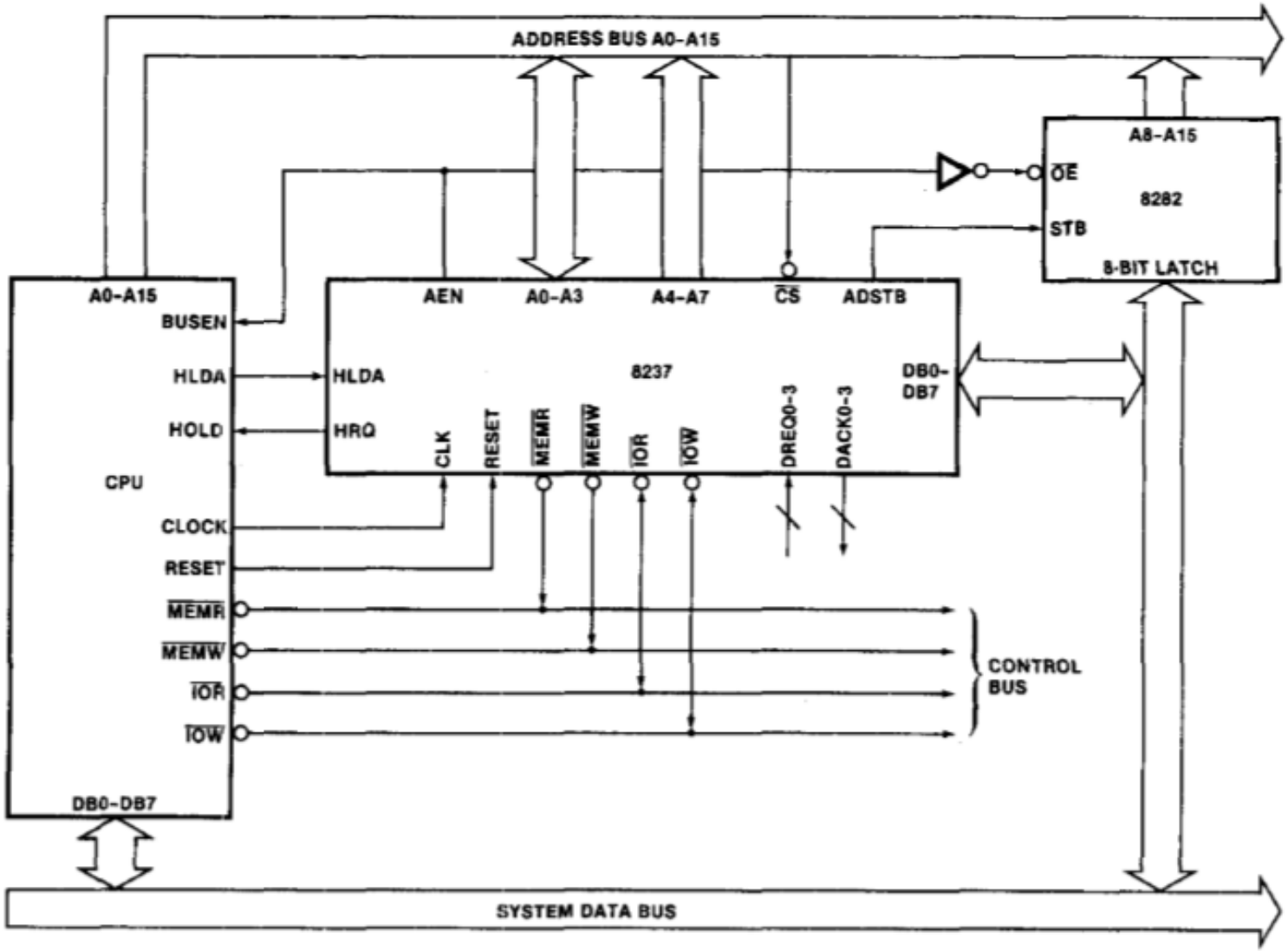
DACK = DMA acknowledge
DREQ = DMA request
HLDA = HOLD acknowledge
HRQ = HOLD request

Figure 7.14 8237 DMA Usage of System Bus

Example: 8237 DMA controller



8237 interface



8237 internal registers

Name	Size	Number
Base Address Registers	16 bits	4
Base Word Count Registers	16 bits	4
Current Address Registers	16 bits	4
Current Word Count Registers	16 bits	4
Temporary Address Register	16 bits	1
Temporary Word Count Register	16 bits	1
Status Register	8 bits	1
Command Register	8 bits	1
Temporary Register	8 bits	1
Mode Registers	6 bits	4
Mask Register	4 bits	1
Request Register	4 bits	1

Figure 3. 8237 Internal Registers

Register	Operation	Signals						
		$\overline{CS}$	$\overline{IOR}$	$\overline{IOW}$	A3	A2	A1	A0
Command	Write	0	1	0	1	0	0	0
Mode	Write	0	1	0	1	0	1	1
Request	Write	0	1	0	1	0	0	1
Mask	Set/Reset	0	1	0	1	0	1	0
Mask	Write	0	1	0	1	1	1	1
Temporary	Read	0	0	1	1	1	0	1
Status	Read	0	0	1	1	0	0	0

Figure 5. Definition of Register Codes

8237 address and word-count register access

Channel	Register	Operation	Signals							Internal Flip-Flop	Data Bus DB0–DB7
			$\overline{CS}$	$\overline{IOR}$	$\overline{IOW}$	A3	A2	A1	A0		
0	Base and Current Address	Write	0	1	0	0	0	0	0	0	A0–A7
			0	1	0	0	0	0	0	1	A8–A15
	Current Address	Read	0	0	1	0	0	0	0	0	A0–A7
			0	0	1	0	0	0	0	1	A8–A15
	Base and Current Word Count	Write	0	1	0	0	0	0	1	0	W0–W7
			0	1	0	0	0	0	1	1	W8–W15
	Current Word Count	Read	0	0	1	0	0	0	1	0	W)0–W7
			0	0	1	0	0	0	1	1	W8–W15
1	Base and Current Address	Write	0	1	0	0	0	1	0	0	A0–A7
			0	1	0	0	0	1	0	1	A8–A15
	Current Address	Read	0	0	1	0	0	1	0	0	A0–A7
			0	0	1	0	0	1	0	1	A8–A15
	Base and Current Word Count	Write	0	1	0	0	0	1	1	0	W0–W7
			0	1	0	0	0	1	1	1	W8–W15
	Current Word Count	Read	0	0	1	0	0	1	1	0	W)0–W7
			0	0	1	0	0	1	1	1	W8–W15
2	Base and Current Address	Write	0	1	0	0	1	0	0	0	A0–A7
			0	1	0	0	1	0	0	1	A8–A15
	Current Address	Read	0	0	1	0	1	0	0	0	A0–A7
			0	0	1	0	1	0	0	1	A8–A15
	Base and Current Word Count	Write	0	1	0	0	1	0	1	0	W0–W7
			0	1	0	0	1	0	1	1	W8–W15
	Current Word Count	Read	0	0	1	0	1	0	1	0	W)0–W7
			0	0	1	0	1	0	1	1	W8–W15
3	Base and Current Address	Write	0	1	0	0	1	1	0	0	A0–A7
			0	1	0	0	1	1	0	1	A8–A15
	Current Address	Read	0	0	1	0	1	1	0	0	A0–A7
			0	0	1	0	1	1	0	1	A8–A15
	Base and Current Word Count	Write	0	1	0	0	1	1	1	0	W0–W7
			0	1	0	0	1	1	1	1	W8–W15
	Current Word Count	Read	0	0	1	0	1	1	1	0	W)0–W7
			0	0	1	0	1	1	1	1	W8–W15

Figure 7. Word Count and Address Register Command Codes

8237 simplified operation (1)

- CPU commands DMA module to do data transfer.
  - Set up: CPU writes 8237's command register
  - Direction of the transfer: CPU writes 8237's mode register
  - Initial memory address: CPU writes base and current address registers (in 2 successive bytes: 8237 has only 8-bit data port)
  - Number of words to transfer: CPU writes base and current word count registers

8237 simplified operation (1)

- CPU commands DMA module to do data transfer.
  - I/O address: 8237 does not distinguish I/O address.
    $\Rightarrow$ Single 8237 can take up to 4 devices.
    - each device pairs 8237 on a dedicated channel.
    - channel 0-3: $DREQ0 - DREQ3$ and $DACK0 - DACK3$
    - 8237 uses $DACK0 - DACK3$ to notify the appropriate device
    - note: 8237 can be set up in cascading configuration to take more than 4 devices

$\Rightarrow$ CPU can initiate transfer by writing 8237's request register

$\Rightarrow$ or let the I/O initiate the DMA by writing 8237's mask register

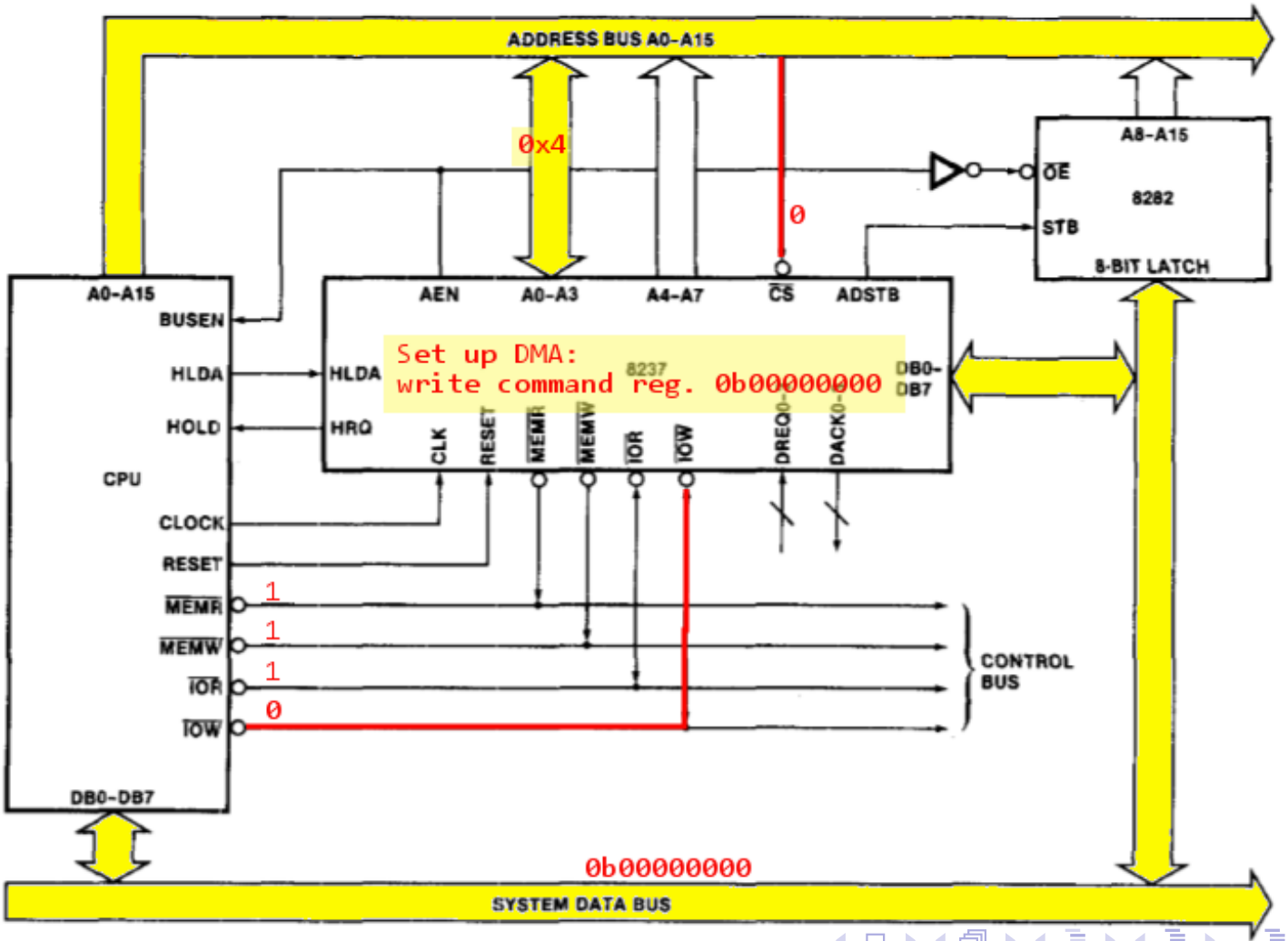
- I/O device signals *DREQ* to 8237 and 8237 answers with *DACK*

8237 simplified operation (2)

- CPU commands DMA module to do data transfer.
- CPU continues with other work,
  while DMA module transfers the data.
  - Before starting transfer, 8237 requests the bus holding right (*HRQ*)
  - CPU grants bus holding to 8237 (*HLDA*)
  - 8237 keeps *AEN* on to signal its right on the bus
    (and also to enable latch 8282 output)
  - 8237 starts data transfer and does it until reaching zero count
- Once the transfer is complete, DMA module sends an interrupt signal to CPU:
  - 8237 provide $\overline{EOP}$ to signal the end of process
  - and CPU can monitor the status through reading 8237's status register.

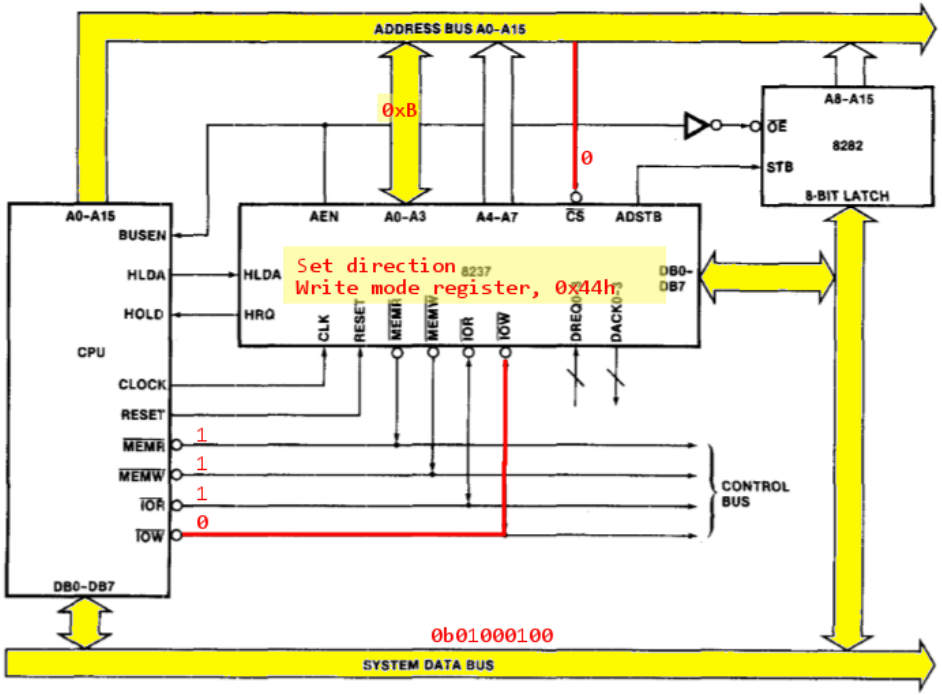
8237 example timing (1)

Set up

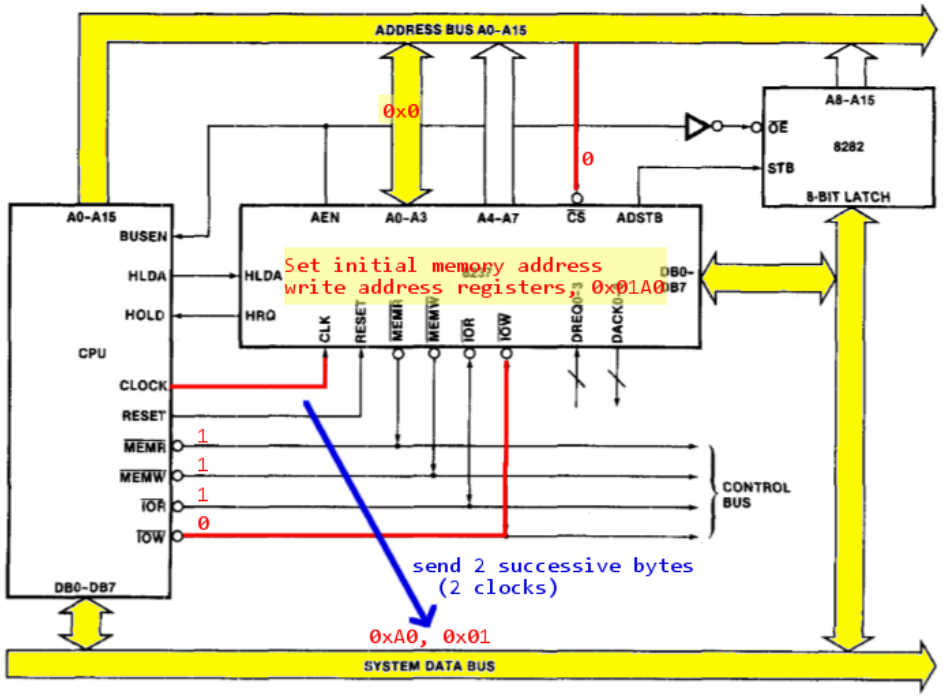


8237 example timing (2)

Channel-0 direction: write transfer (I/O → memory)



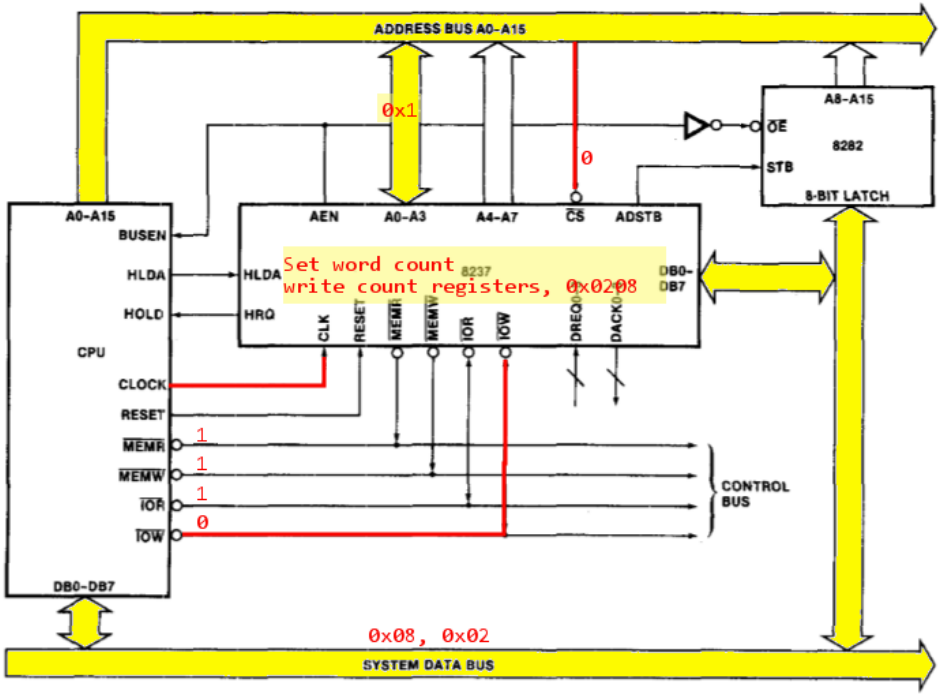
Channel-0 initial address: 0x01A0



8237 example timing (3)

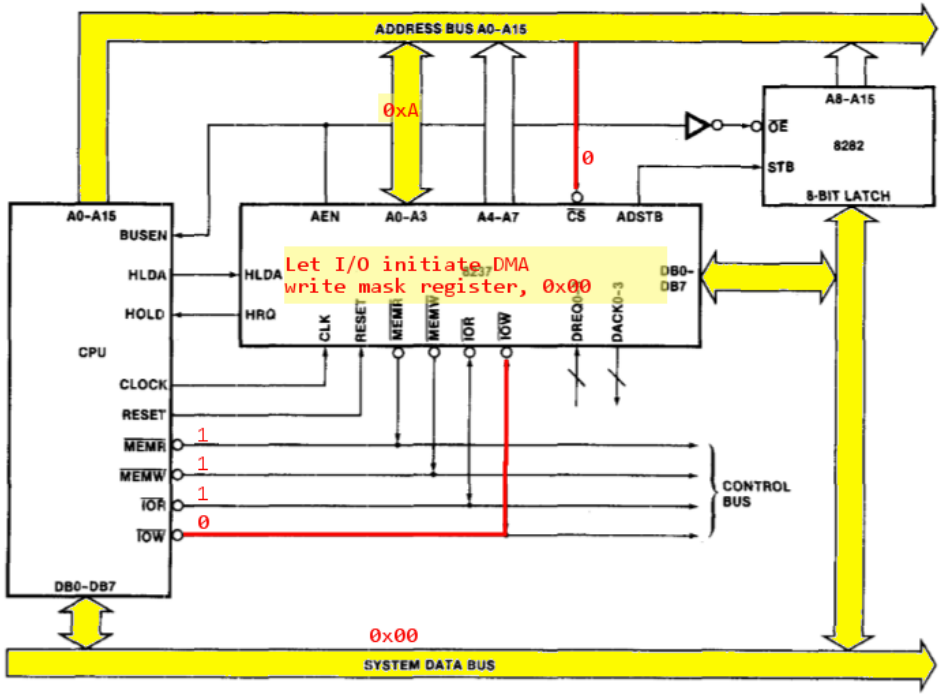
Channel-0 word count:

0x0208 = 520



Initiation:

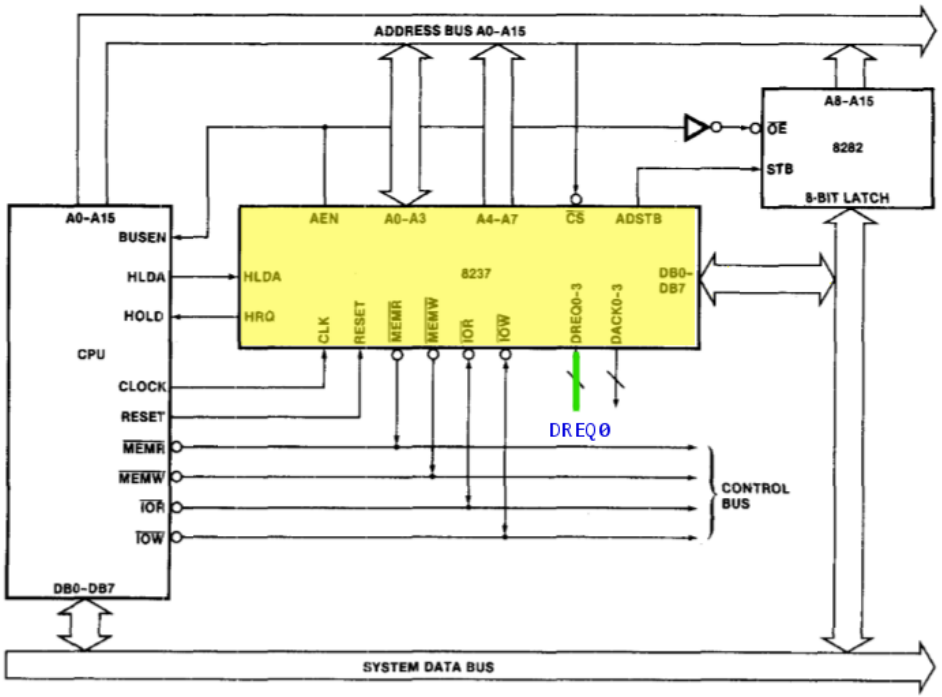
Any channel can initiate DMA

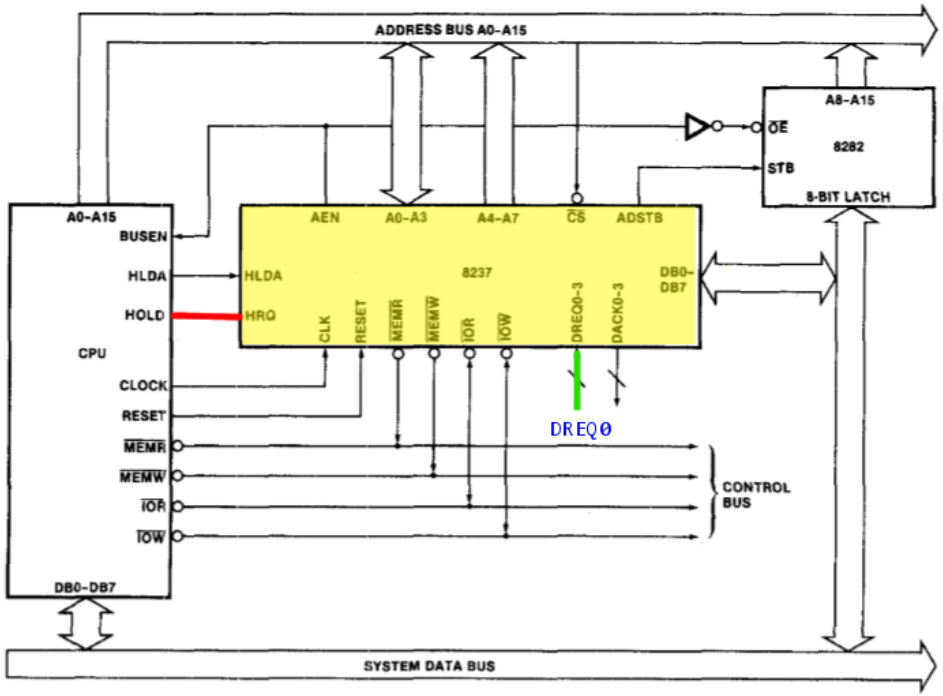


8237 example timing (4)

DMA operation

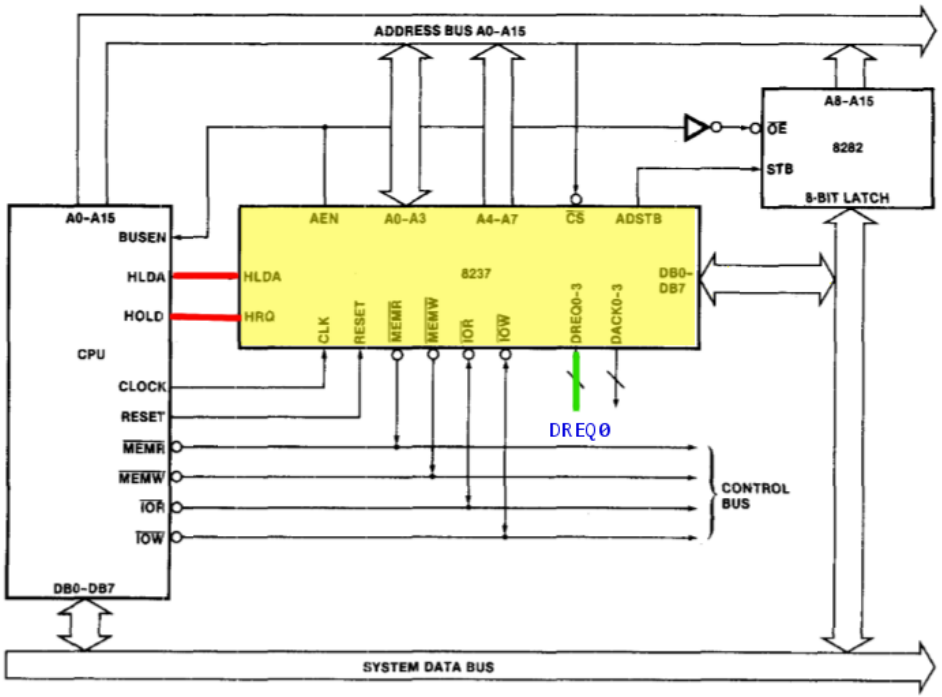
DMA is requested by dev 0: *DREQ*0. 8237 requests bus holding right.



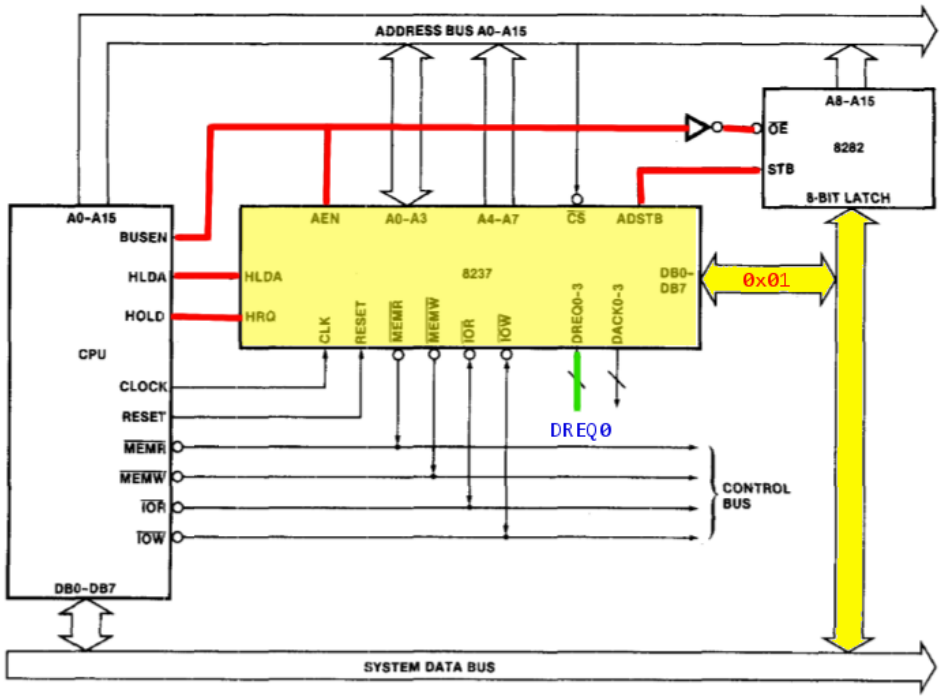


8237 example timing (5)

CPU grants 8237 the bus holding.
(CPU cannot use the bus,
until 8237 releases)



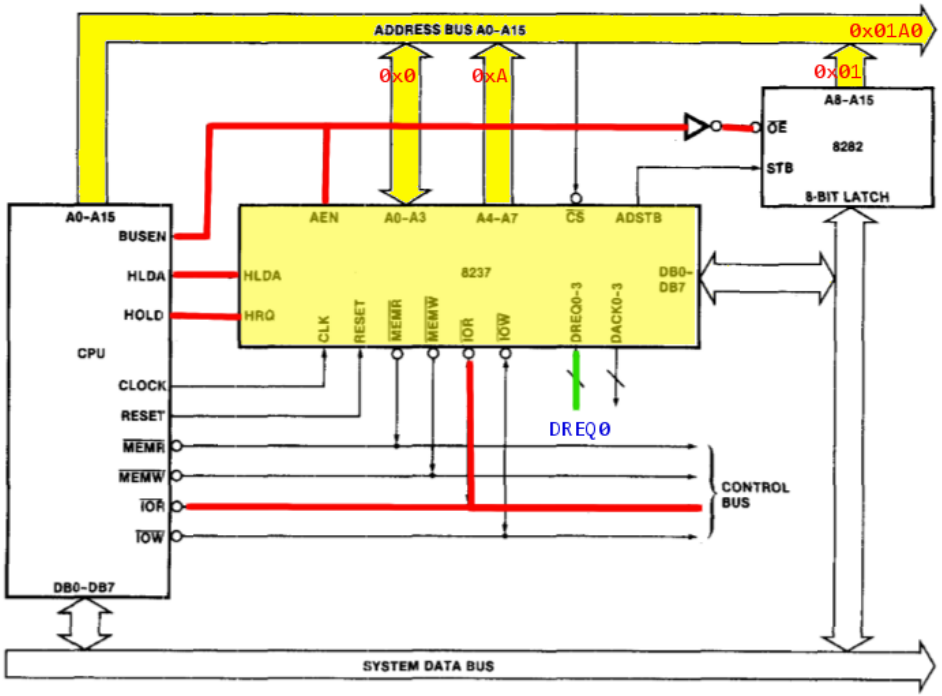
8237 holds the bus
and latches the upper address bytes.
(*AEN* $\rightarrow$ *BUSEN* is to hold the bus)



8237 example timing (6)

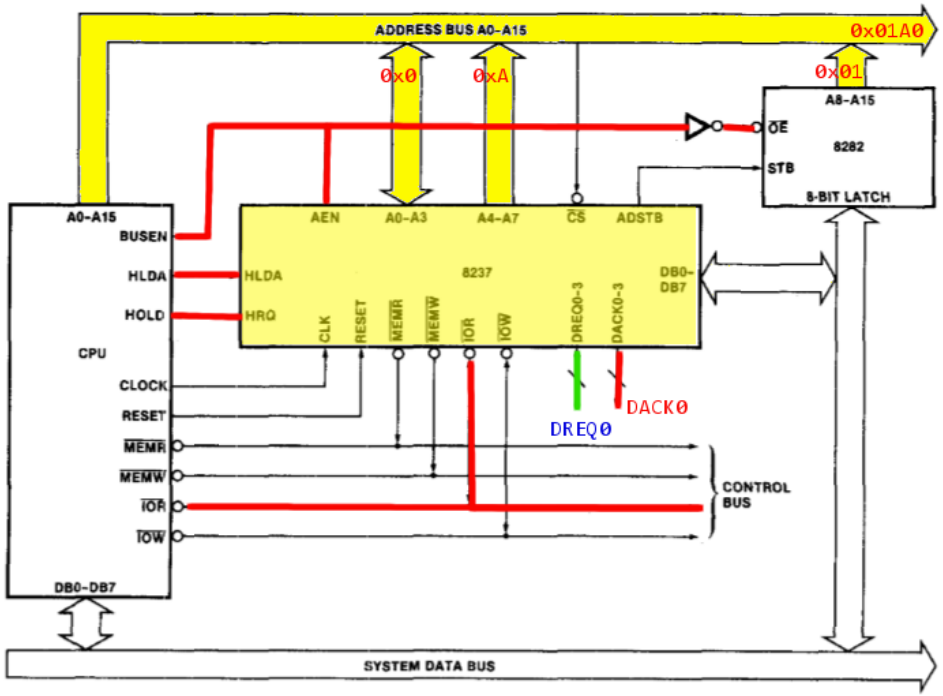
Read I/O.

(Note: $\overline{IOR}$ can be seen by all devices)



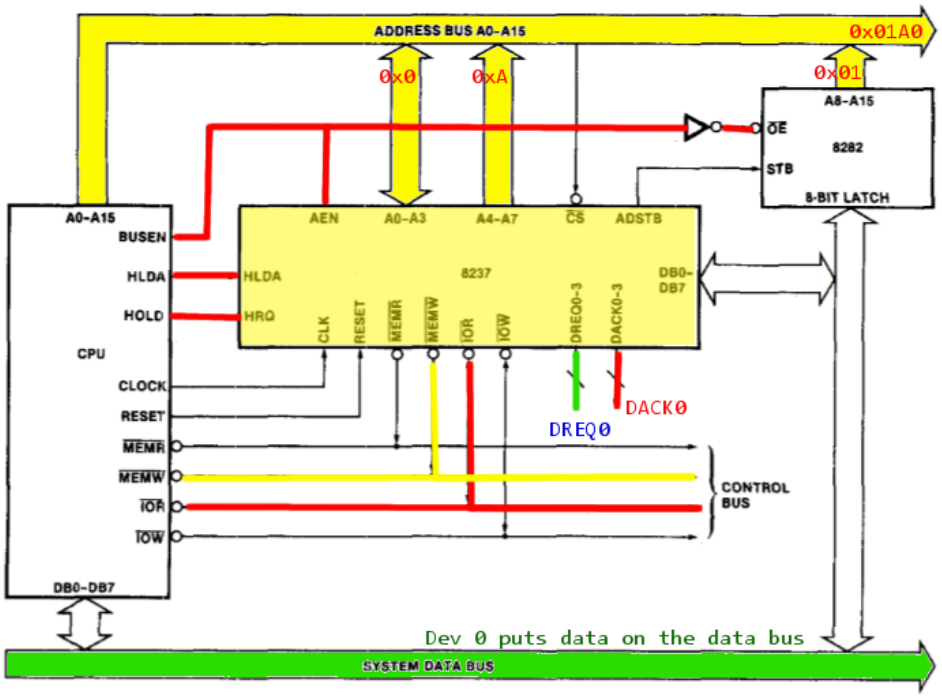
Notify the device.

(Signal *DACK*0 lets the device know which is chosen)

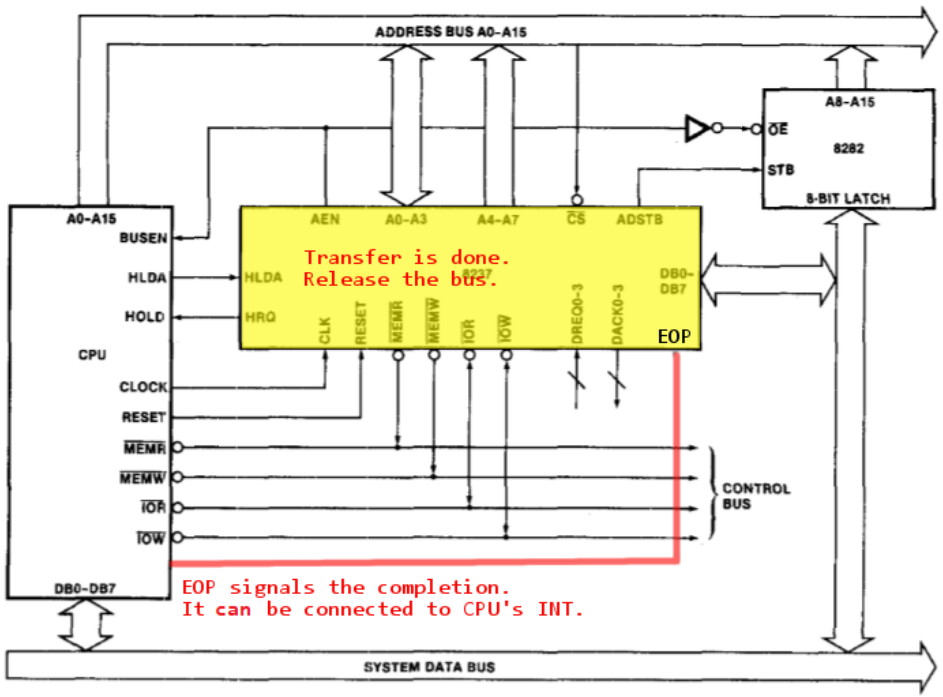


8237 example timing (7)

Write memory.
(Data is already on the bus.
Ask memory to take it in.
8237 does not need to buffer.)



8237 keeps doing the transfer,
until the count gets to 0.
Once it's done,
update status reg and raise $\overline{EOP}$.

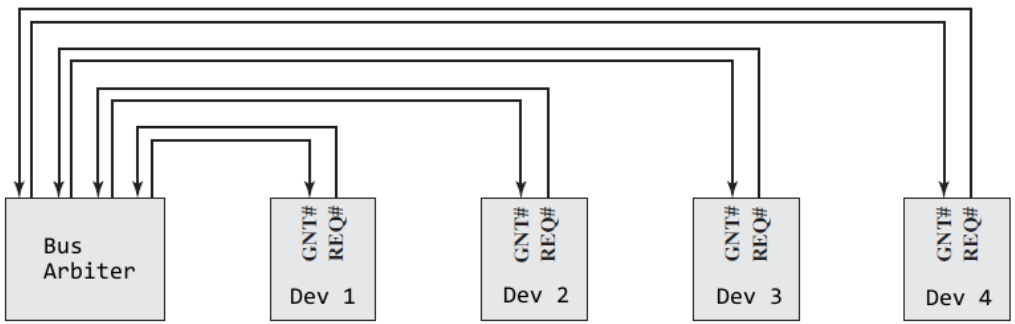


Bus arbitration

- Allowing DMA, I/O device can read or write data directly to memory.
- It is possible to have more than one module asking to use the bus.

  $\Rightarrow$ arbitation is to determine which device will be granted the bus master.

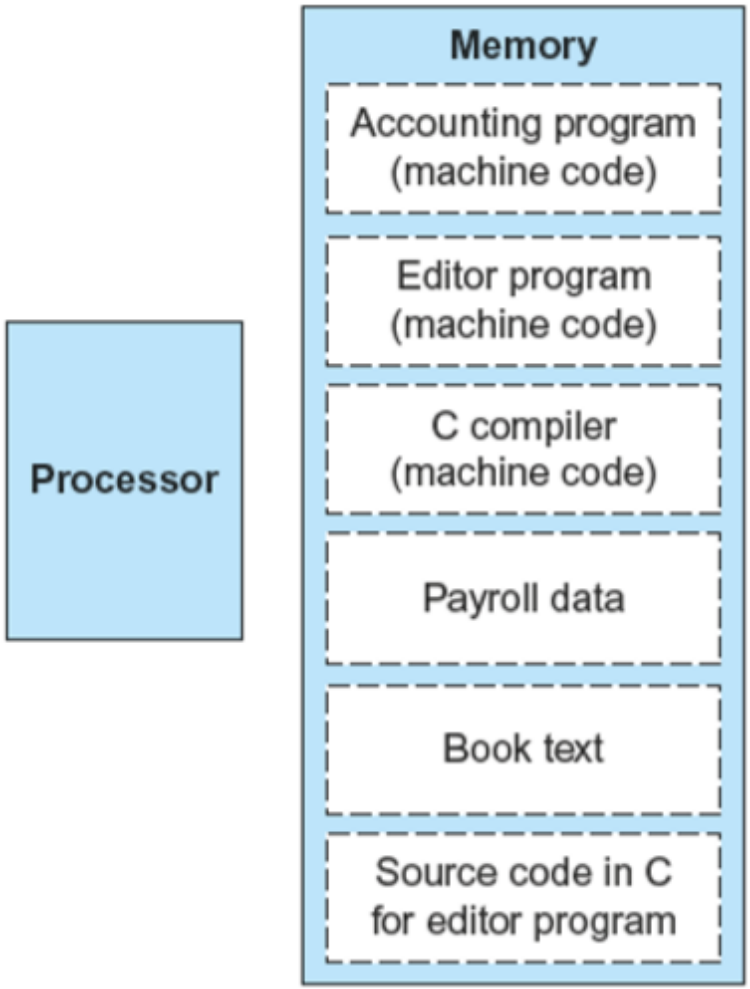
  $\Rightarrow$ Centralized bus arbitration:
  - Bus controller (or bus arbiter) allocates time on the bus.
  - Bus controller may be a separate module or a part of the processor.
  - Bus request and grant signals are transmitted through dedicated lines between each module and the bus controller.



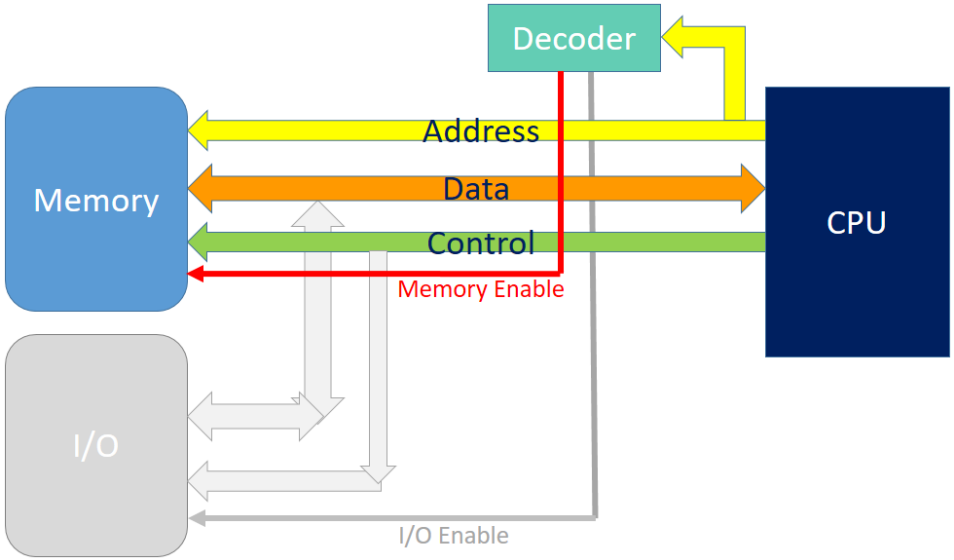
Stored-program concept

Stored-program concept:
instructions and data of many types can be stored in memory as numbers, leading to the stored-program computer.

- Instructions are represented as numbers
- Programs are stored in memory to be read and write, just like data



Memory

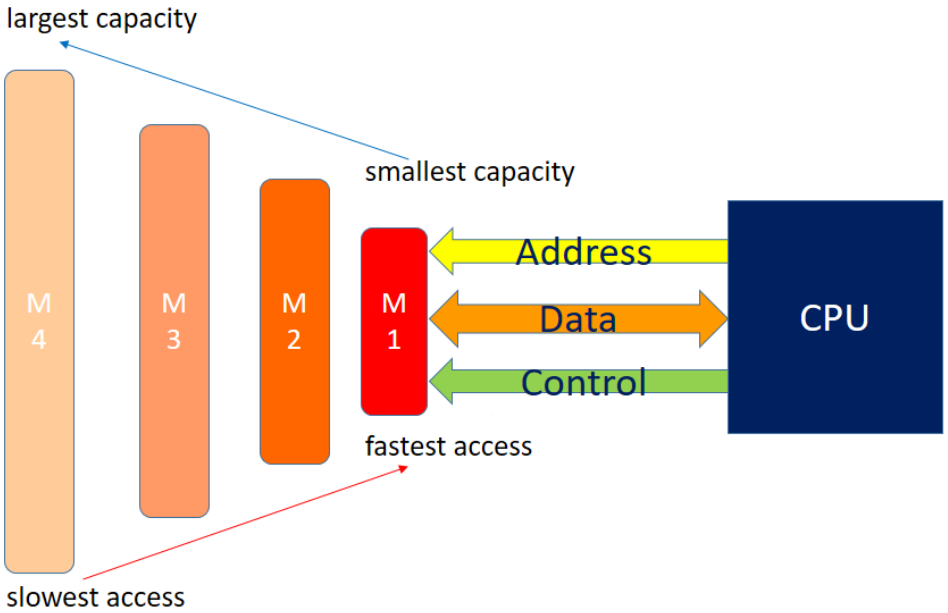


Memory is better to be

- Fast
- Large
- Affordable

$\Rightarrow$ Memory hierarchy

Memory hierarchy



Memory hierarchy

- A faster memory is closer to CPU.
- A slower memory is farther from CPU.
- A slower memory has larger capacity.
- $\equiv$ a larger memory is farther from CPU.

Memory hierarchy: cache

Characteristic	ARM Cortex-A8	Intel Nehalem
L1 cache organization	Split instruction and data caches	Split instruction and data caches
L1 cache size	32 KiB each for instructions/data	32 KiB each for instructions/data per core
L1 cache associativity	4-way (I), 4-way (D) set associative	4-way (I), 8-way (D) set associative
L1 replacement	Random	Approximated LRU
L1 block size	64 bytes	64 bytes
L1 write policy	Write-back, Write-allocate(?)	Write-back, No-write-allocate
L1 hit time (load-use)	1 clock cycle	4 clock cycles, pipelined
L2 cache organization	Unified (instruction and data)	Unified (instruction and data) per core
L2 cache size	128 KiB to 1 MiB	256 KiB (0.25 MiB)
L2 cache associativity	8-way set associative	8-way set associative
L2 replacement	Random(?)	Approximated LRU
L2 block size	64 bytes	64 bytes
L2 write policy	Write-back, Write-allocate (?)	Write-back, Write-allocate
L2 hit time	11 clock cycles	10 clock cycles
L3 cache organization	–	Unified (instruction and data)
L3 cache size	–	8 MiB, shared
L3 cache associativity	–	16-way set associative
L3 replacement	–	Approximated LRU
L3 block size	–	64 bytes
L3 write policy	–	Write-back, Write-allocate
L3 hit time	–	35 clock cycles

FIGURE 5.44 Caches in the ARM Cortex-A8 and Intel Core I7 920.

Simple cache control

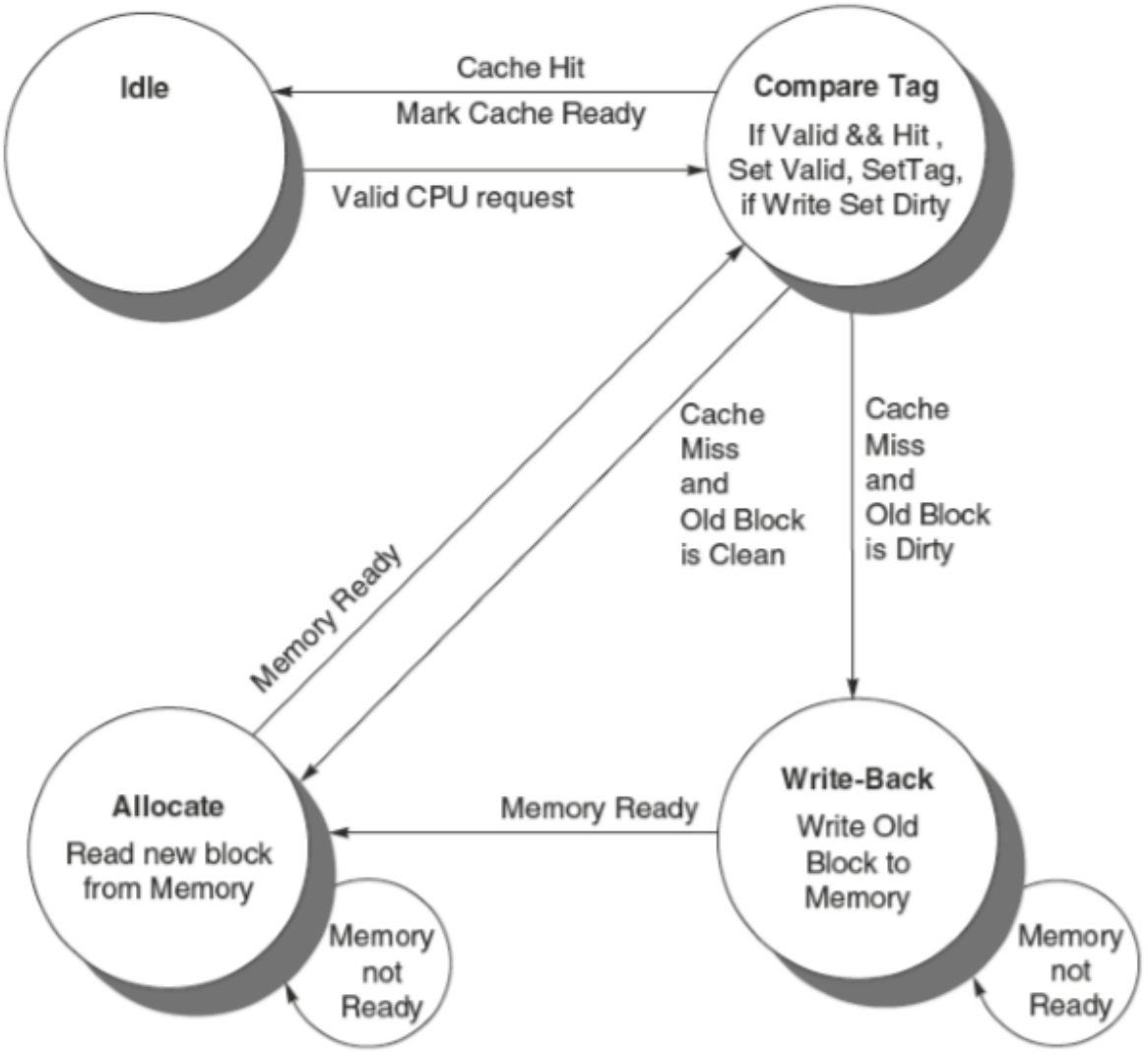


FIGURE 5.40 Four states of the simple controller.

Cache data

Where is data put into cache?

E.g., direct map: assign only one spot it can go.

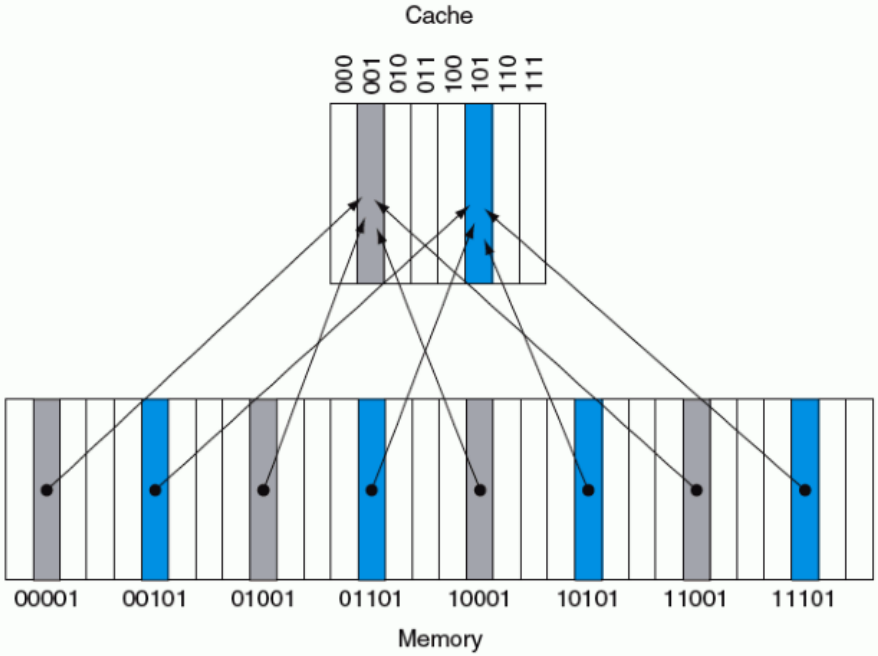


FIGURE 7.5 A direct-mapped cache with eight entries showing the addresses of memory words between 0 and 31 that map to the same cache locations. Because there are eight words in the cache, an address X maps to the cache word X modulo 8. That is, the low-order $\log_2(8) = 3$ bits are used as the cache index. Thus, addresses 00001_{two}, 01001_{two}, 10001_{two}, and 11001_{two} all map to entry 001_{two} of the cache, while addresses 00101_{two}, 01101_{two}, 10101_{two}, and 11101_{two} all map to entry 101_{two} of the cache.

Cache-memory

How is memory address associated to cache location?

E.g., index table

Index	V	Tag	Dlata
000	N		
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memorlly (10110_{two})
111	N		

c. After handling a miss of address (11010_{two})

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

d. After handling a miss of address (10000_{two})

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

e. After handling a miss of address (00011_{two})

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	10_{two}	Memory (10010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

f. After handling a miss of address (10010_{two})

FIGURE 7.6 The cache contents are shown after each reference request that *misses*, with the index and tag fields shown in binary. The cache is initially empty, with all valid bits (V entry in cache) turned off (N). The processor requests the following addresses: 10110_{two} (miss), 11010_{two} (miss), 10110_{two} (hit), 11010_{two} (hit), 10000_{two} (miss), 00011_{two} (miss), 10000_{two} (hit), and 10010_{two} (miss). The figures show the cache contents after each miss in the sequence has been handled. When address 10010_{two} (18) is referenced, the entry for address 11010_{two} (26) must be replaced, and a reference to 11010_{two} will cause a subsequent miss. The tag field will contain only the upper portion of the address. The full address of a word contained in cache block i with tag field j for this cache is $j \times 8 + i$, or equivalently the concatenation of the tag field j and the index i. For example, in cache f above, index 010 has tag 10 and corresponds to address 10010.

Cache: read data

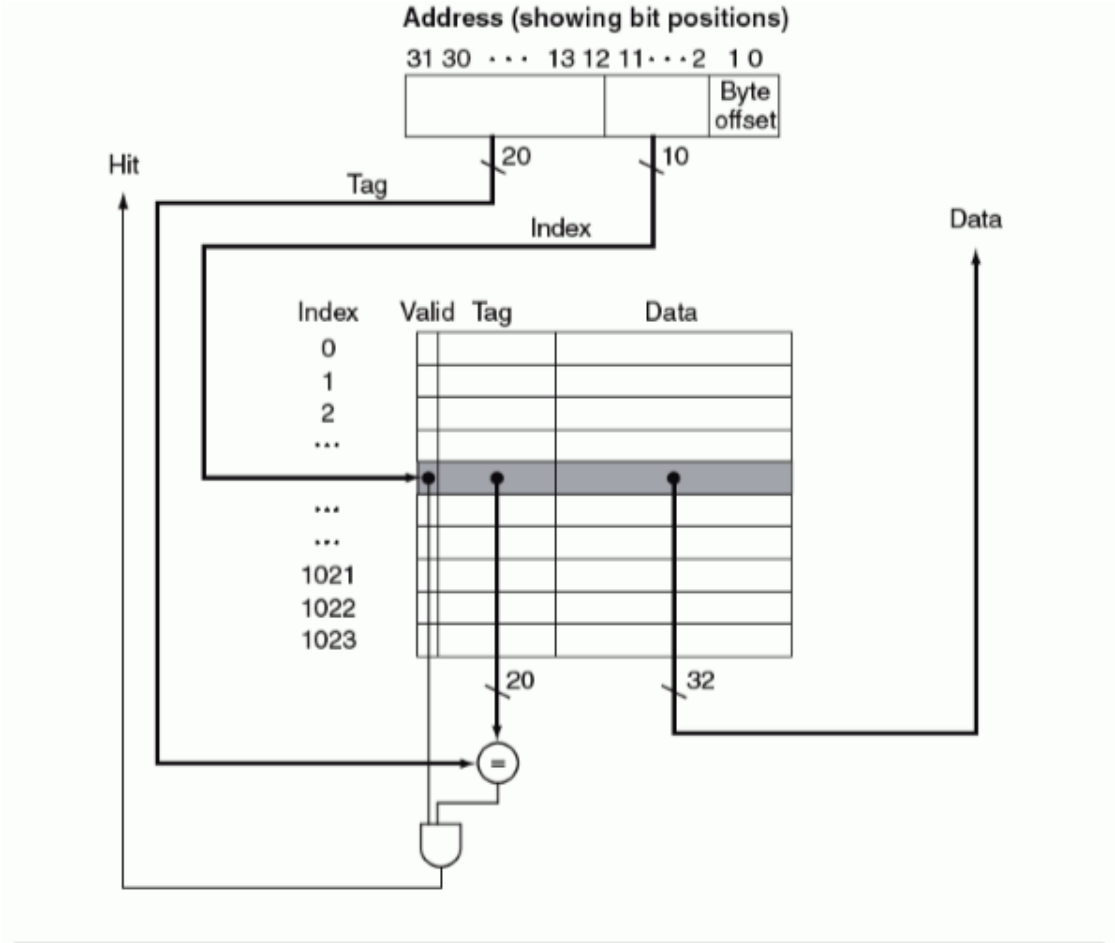


FIGURE 7.7 For this cache, the lower portion of the address is used to select a cache entry consisting of a data word and a tag. The tag from the cache is compared against the upper portion of the address to determine whether the entry in the cache corresponds to the requested address. Because the cache has 2^{10} (or 1024) words and a block size of 1 word, 10 bits are used to index the cache, leaving 32 – 10 – 2 = 20 bits to be compared against the tag. If the tag and upper 20 bits of the address are equal and the valid bit is on, then the request hits in the cache, and the word is supplied to the processor. Otherwise, a miss occurs.

Cache: handling miss

If there is a miss, stall the process and

- (1) send the missed address to memory,
- (2) read memory and wait for data,
- (3) put data into the corresponding cache entry, write cache tag and valid bit,
- (4) and repeat the process (this time data will be found in cache).

Cache: replacement policies

- When miss, data must be fetched from memory.
- If there is an empty spot, put it there.
- If no empty spot, where is the new data put?
  E.g., LRU (Least Recently Used)

• Note: in practice, it is more efficient to fetch data in block (rather than a single word).
E.g., FastMath uses a 16-word block.

Cache: handling write

- Issue of Consistency
- schemes
  - write-through: update both cache and memory
  - write buffer: write to cache and buffer and move on, while writing from buffer to memory
  - write-back: update cache and an extra flag, e.g., Dirty, and write the dirty cache block back to memory when it is replaced.

Cache: write miss

When address to write is not in cache ("write miss"):

- Write allocate (or "fetch on write")
  Fetch data into cache, data at the missed-write location is loaded to cache, followed by a write-hit operation.
- No-write allocate (or "write-no-allocate" or "write around")
  Write directly to the downstream memory.

FastMath's cache (FastMath–MIPS microprocessor)

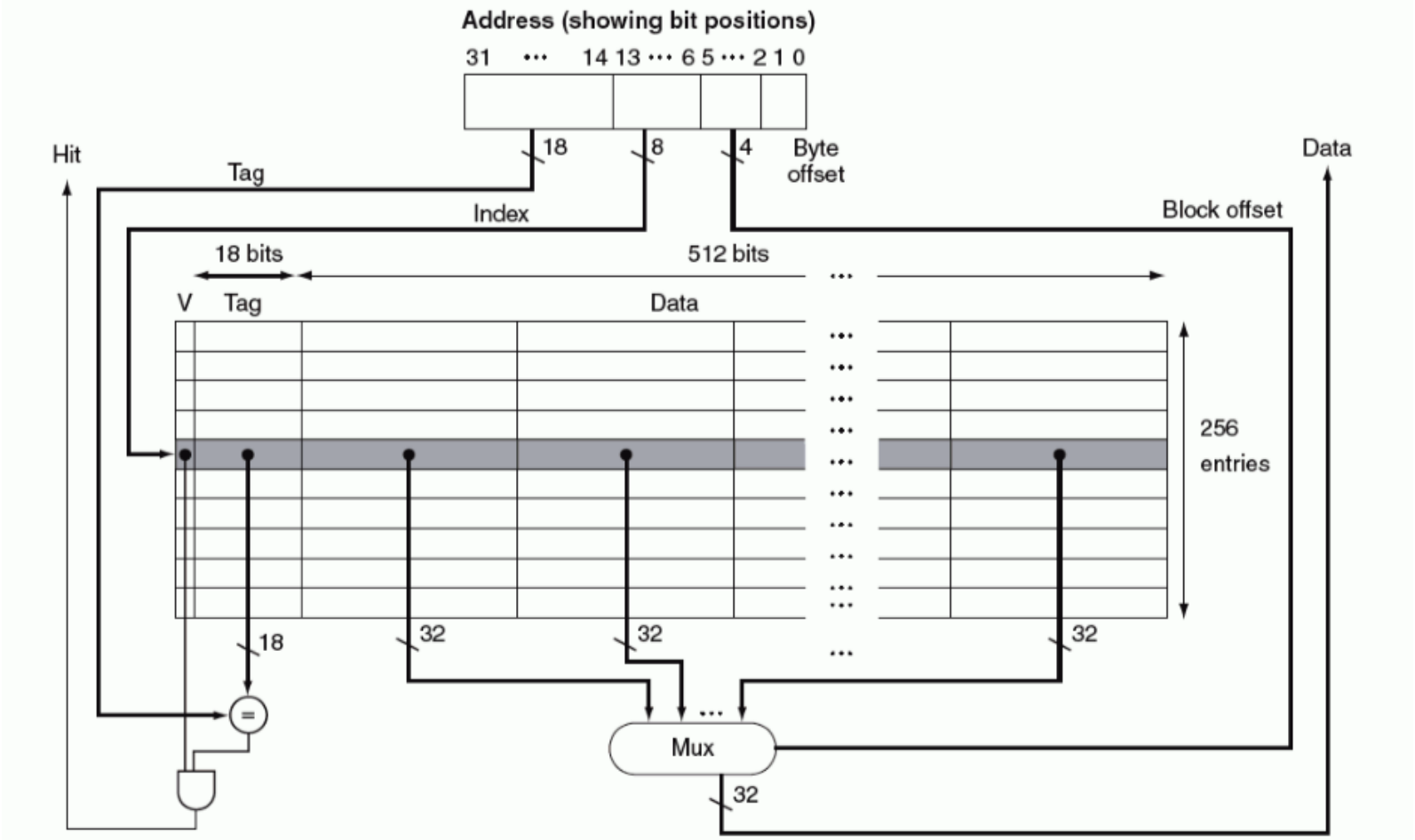


FIGURE 7.9 The 16 KB caches in the Intrinsity FastMATH each contain 256 blocks with 16 words per block. The tag field is 18 bits wide and the index field is 8 bits wide, while a 4-bit field (bits 5–2) is used to index the block and select the word from the block using a 16-to-1 multiplexor. In practice, to eliminate the multiplexor, caches use a separate large RAM for the data and a smaller RAM for the tags, with the block offset supplying the extra address bits for the large data RAM. In this case, the large RAM is 32 bits wide and must have 16 times as many words as blocks in the cache.

Cache associativity

Where is data put into cache? Associativity.

Direct map: rigid, but least time to search

Full association: flexible, but max time to search

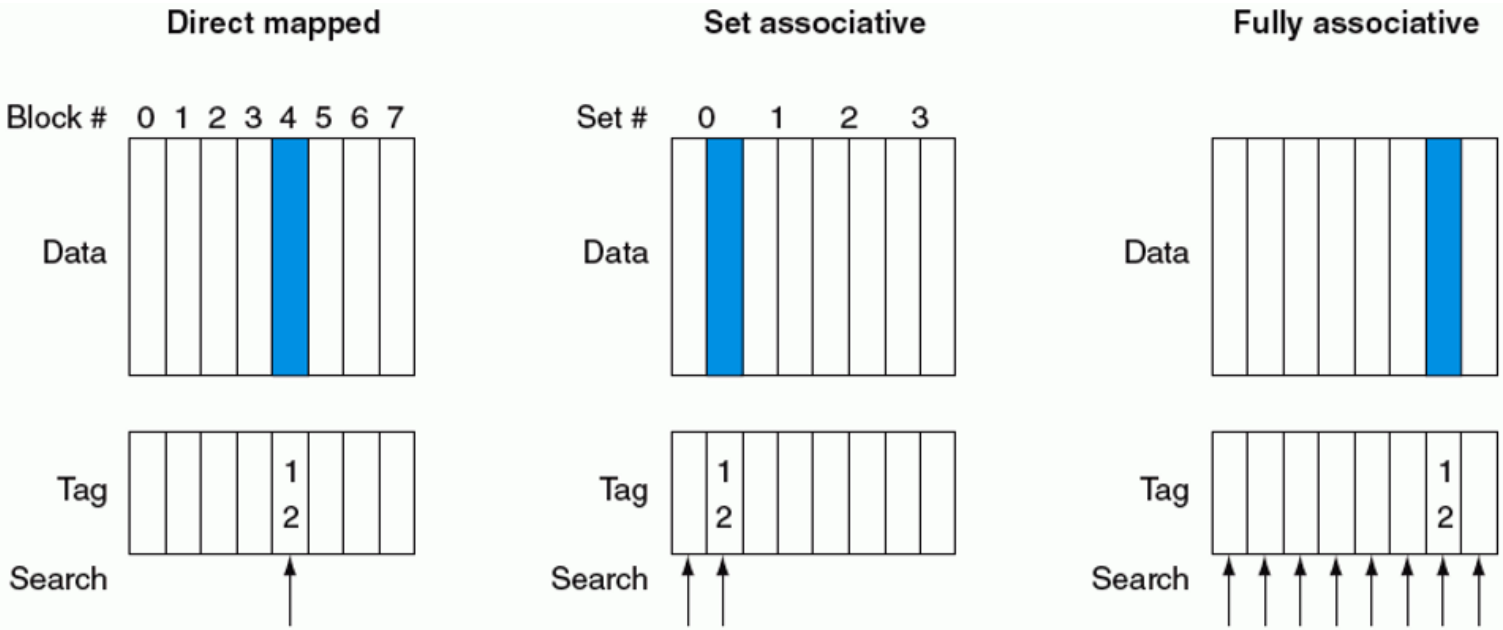


FIGURE 7.13 The location of a memory block whose address is 12 in a cache with 8 blocks varies for direct-mapped, set-associative, and fully associative placement. In direct-mapped placement, there is only one cache block where memory block 12 can be found, and that block is given by (12 modulo 8) = 4. In a two-way set-associative cache, there would be four sets, and memory block 12 must be in set (12 mod 4) = 0; the memory block could be in either element of the set. In a fully associative placement, the memory block for block address 12 can appear in any of the eight cache blocks.

Materials for bootstrapping

- Bovet and Cesati, Understanding Linux Kernel 3rd Ed., O'Reilly, 2006. Appendix A: System Start Up.
- Russinovich, Solomon, and Lonescu, Windows Internals 6th Ed., Part II, Microsoft Press 2012. Chapter 13: Startup and Shutdown.

Bootstrapping

- Boot a computer $\equiv$ bring a portion of OS into memory and have the processor execute it.
- Tedious, long task.
- Highly hardware dependent.
- Computer Bootstrapping
  - Basic Input Output System (BIOS)
  - Extensible Firmware Interface (EFI)

Basic Input Output System (BIOS)

- This is specific to 80x86 system.
- Boot $\Rightarrow$ the mainboard asserts RESET signal to the CPU.
- $\Rightarrow$ EIP ($\equiv$ Program Counter) is set to 0xFFFFFFF0.
- Address 0xFFFFFFF0 is mapped to Read-Only Memory (ROM) or flash memory, which stores a program, called Basic Input/Output System (BIOS).
  - BIOS includes several interrupt-driven low-level procedures (used by all OSs in the booting phase) to handle the hardware devices.
  - BIOS program, e.g., AMI and Phoenix, is proprietary.

  v.s. opensource project, i.e., CoreBoot.

BIOS program

BIOS program:

- 1. Run a series of tests on the computer hardware

  ⇒ to check which devices are present and if they are working properly.

  ⇒ This is called “Power-On Self-Test” (POST).

- 2. Initialize the hardware devices.
- 3. Search for an OS to boot.

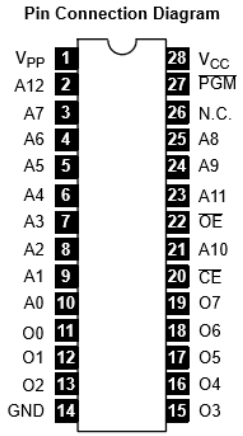
  ⇒ depending on the BIOS setting, the procedure may try to access the first sector (boot sector) of hard disk and then CD-ROM in the system.

- 4. Once a valid device is found, copy the contents of the boot sector into RAM, starting from physical address 0x00007c00.
- 5. Jump to address 0x00007c00 to execute the code just loaded.

Example ROM, 2764 Erasable EPROM 8K x 8

Erasable EPROM 8K x 8

- Ultraviolet Erasable
  and Electrically Programmable
- Access Time: 250ns Max



Programming:

Programming begins with erasing all data and consequently having all bits in the high (1) level state. Data is then entered by programming a low (0) level TTL signal into the chosen bit location.

The NTE2764 is placed in programming mode by applying a low (0) level TTL signal to the $\overline{CE}$ and $\overline{PGM}$ with V_{pp} at +21V. The data to be programmed is applied to the output pins in 8–bit parallel form at TTL levels.

Any location can be programmed at any time, either individually, sequentially or at random.

When multiple NTE2764s are connected in parallel except for $\overline{CE}$, individual NTE2764 can be programmed by applying a low (0) level TTL pulse to the $\overline{PGM}$ input of the desired NTE2764 to be programmed.

Programming of multiple NTE2764s in parallel with the same data is easily accomplished. All the like inputs are tied together and programmed by applying a low (0) level TTL pulse to the $\overline{PGM}$ inputs.

Virtual Memory

- to allow a efficient and safe sharing of memory among multiple programs

  $\Rightarrow$ we would like to compile each program into its own address space

  $\Rightarrow$ virtual memory implements translation of program's address space to physical addresses.

  $\Rightarrow$ enforce protection of the program's address space from other programs
- to use memory as a "cache" for secondary storage $\Rightarrow$ to allow a program to exceed the size of primary memory).

Virtual Memory

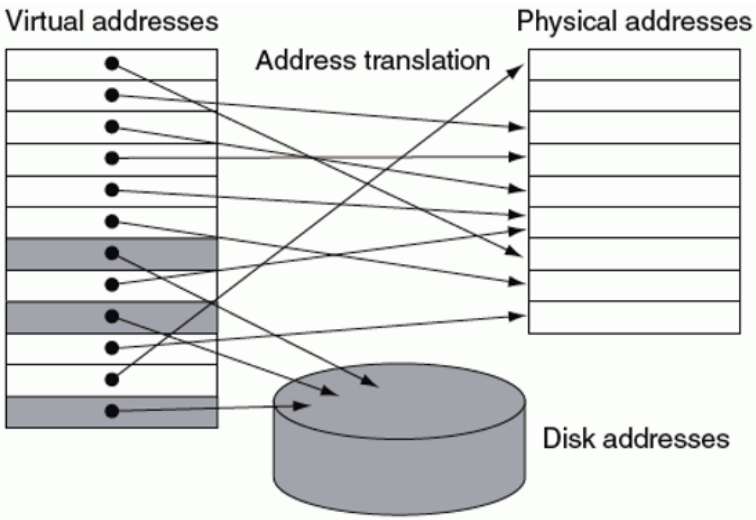
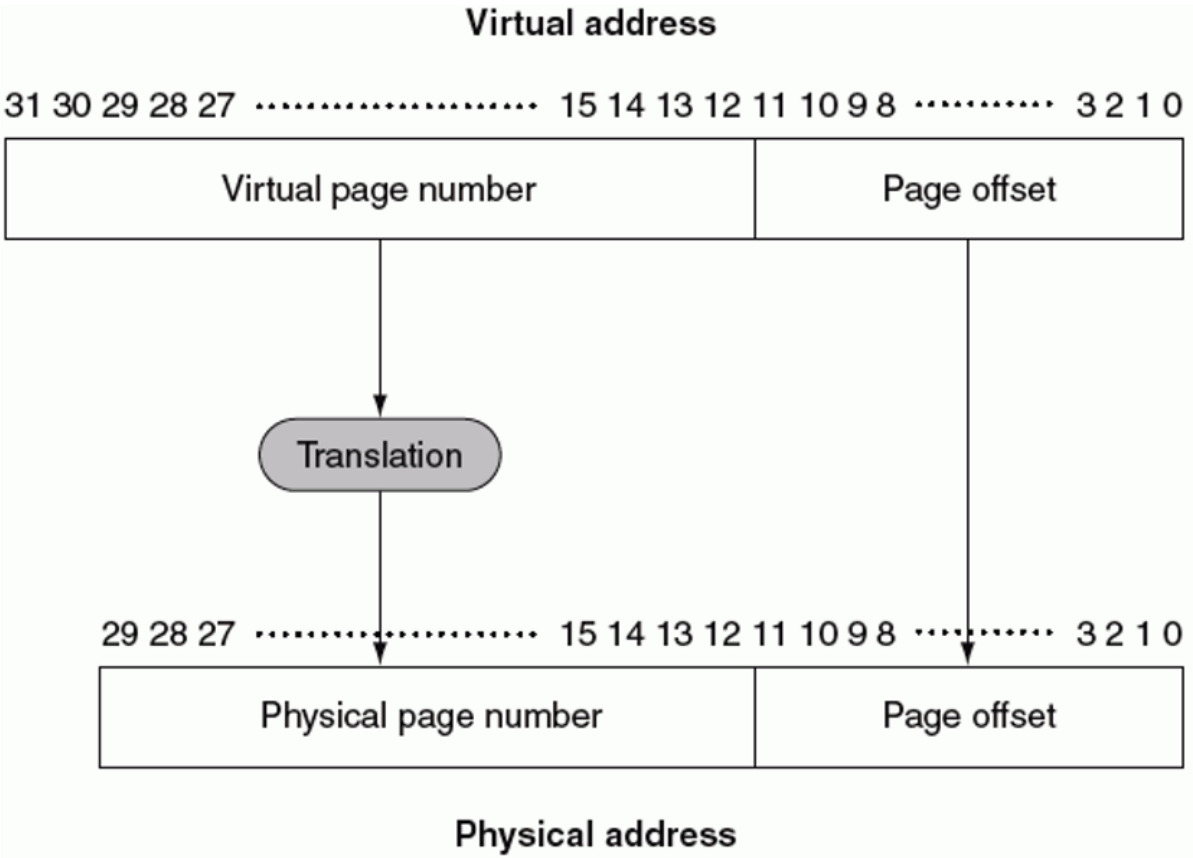


FIGURE 7.19 In virtual memory, blocks of memory (called *pages*) are mapped from one set of addresses (called *virtual addresses*) to another set (called *physical addresses*). The processor generates virtual addresses while the memory is accessed using physical addresses. Both the virtual memory and the physical memory are broken into pages, so that a virtual page is really mapped to a physical page. Of course, it is also possible for a virtual page to be absent from main memory and not be mapped to a physical address, residing instead on disk. Physical pages can be shared by having two virtual addresses point to the same physical address. This capability is used to allow two different programs to share data or code.

- Physical addresses = addresses that memory sees
- Virtual addresses = addresses that a program sees
- similar to cache idea, but different terms
  - virtual memory block = "page"
  - virtual memory miss = "page fault"

Virtual Memory: Address Translation



- Virtual addresses allow access to a larger space than physical memory.

Virtual Memory: Page Table

- To find a physical page,
  $\Rightarrow$ virtual memory uses a page table (in memory)
  - Each program has its own page table
  - page table: virtual page number $\mapsto$ physical page number
  - hardware usually provides a register to point to a page table
    $\Rightarrow$ page table register
  - page table, PC, and registers specify state of the program ("process state")
    $\Rightarrow$ to save a process (to allow other process to run), page table, PC, registers must be saved.

Virtual Memory: Page Fault

- When page fault occurs, OS must find the page in the next level of the hierarchy (e.g., secondary storage) and once the page is obtained, put the page into the main memory
- OS usually creates a space on disk for all the pages of a process when it creates the process.
  $\Rightarrow$ the disk space is called "swap space"
- When there is a page fault, OS will find the page, decide which page on memory it will replace, put the requested page on the memory, and remove the old page and write the old page on the swap space, as well as keep track of which is where.

Virtual Memory: Page Fault

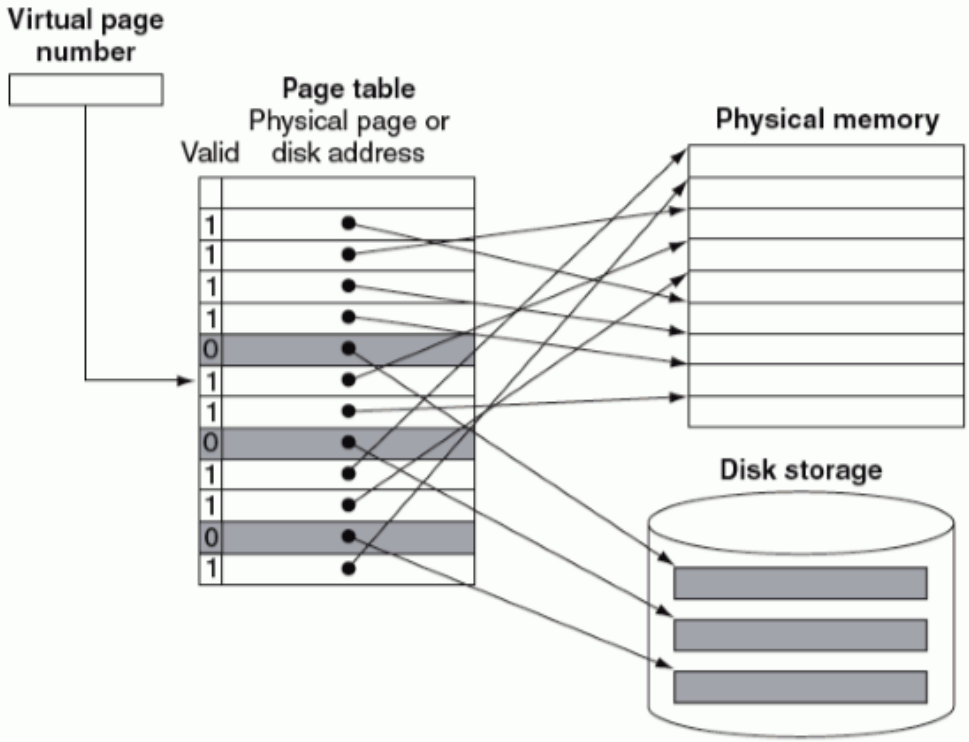


FIGURE 7.22 The page table maps each page in virtual memory to either a page in main memory or a page stored on disk, which is the next level in the hierarchy. The virtual page number is used to index the page table. If the valid bit is on, the page table supplies the physical page number (i.e., the starting address of the page in memory) corresponding to the virtual page. If the valid bit is off, the page currently resides only on disk, at a specified disk address. In many systems, the table of physical page addresses and disk page addresses, while logically one table, is stored in two separate data structures. Dual tables are justified in part because we must keep the disk addresses of all the pages, even if they are currently in main memory. Remember that the pages in main memory and the pages on disk are identical in size.

Virtual Memory: Page Table

Virtual Page Number	Physical Page Number	Valid	Modified
00000	0050	1	0
00001	08A0	1	1
00002	69B2	0	0
:	:	:	:

(รายละเอียดขึ้นกับ OS)

- Virtual Memory อนุญาติให้ program อ้างถึง address space ที่ใหญ่กว่า physical memory ได้

  E.g., VPN 0x00000 = PPN 0x0050 ใน memory และ page ยังไม่ถูกเขียน
  จำนวนบิตของ VPN อาจมากกว่า PPN ได้

- Page table จะเก็บสถานะเมื่อ page ถูกเขียนไว้ เพื่อสามารถทำการ copy back (เขียนกลับ) ได้ถูกต้องกับ page ที่จะถูกนำออกจาก memory (เพื่อแทนด้วย page ใหม่ที่ถูกเรียก)
  E.g., Modified Bit จะระบุว่า page ถูกเขียนหรือไม่
  VPN 0x00001 = PPN 0x08A0 ใน memory และ page ถูกเขียนแล้ว

- Valid Bit จะบอกว่า memory มี page อยู่หรือไม่ ถ้าไม่มี page นั้นจะถูกใน secondary storage (เช่น HDD) และ PPN entry จะเก็บ location ใน secondary storage ไว้
  E.g., VPN 0x00002 ไม่มี page ใน memory
  $\Rightarrow$ ข้อมูลอยู่ swap space ใน HDD: LBA 0x69B2 ($\approx$ header 13, cylinder 26, sector 32)

Roadmap

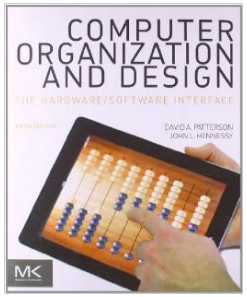
- Computer 101A (Done)

  Computer system, memory access, I/O access, and interrupt
- Computer 101B (Done)

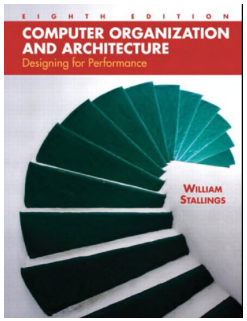
  Data transfer, memory, bootstrapping, and virtual memory
- Instruction set (Next)
- Computer arithmetic
- Data path and control
- Pipeline
- Computer 101C

Reading

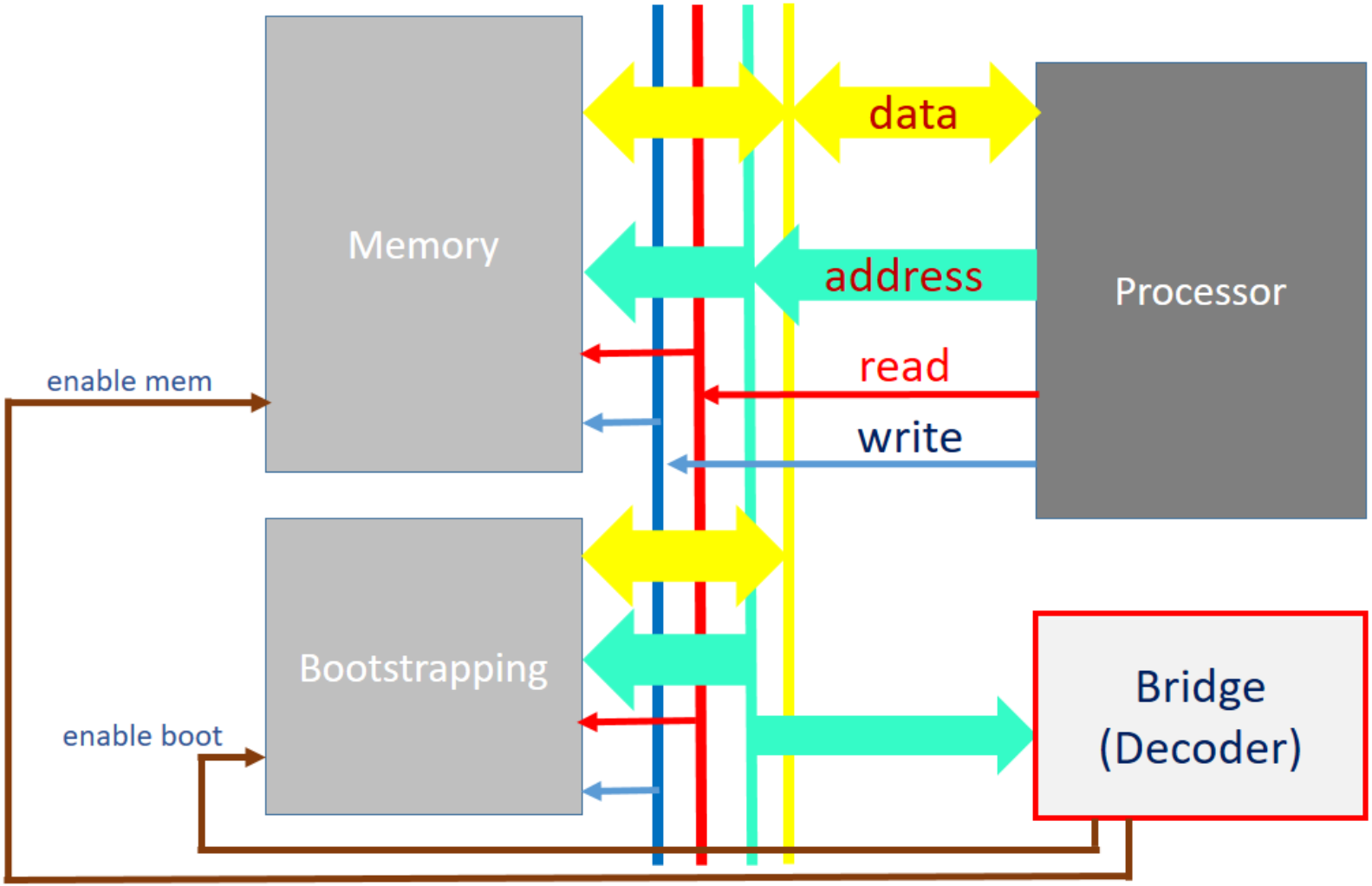
P&H



Stallings



Project (1)


Memory
data
address
Processor
read
write
enable mem
Bootstrapping
Bridge
(Decoder)
enable boot

Project (2)

- Data bus: 32 bits
- Address bus: 32 bits
- Bootstrapping address map:
- Main memory address map:

Project (3)

- Bridge

  Address space
- Memory module

  Size and memory chips
- Bootstrapping module

  Size and chips
- Processor: ALU

  Additional instructions
- Processor: Multi-cycle machine

  Additional instructions